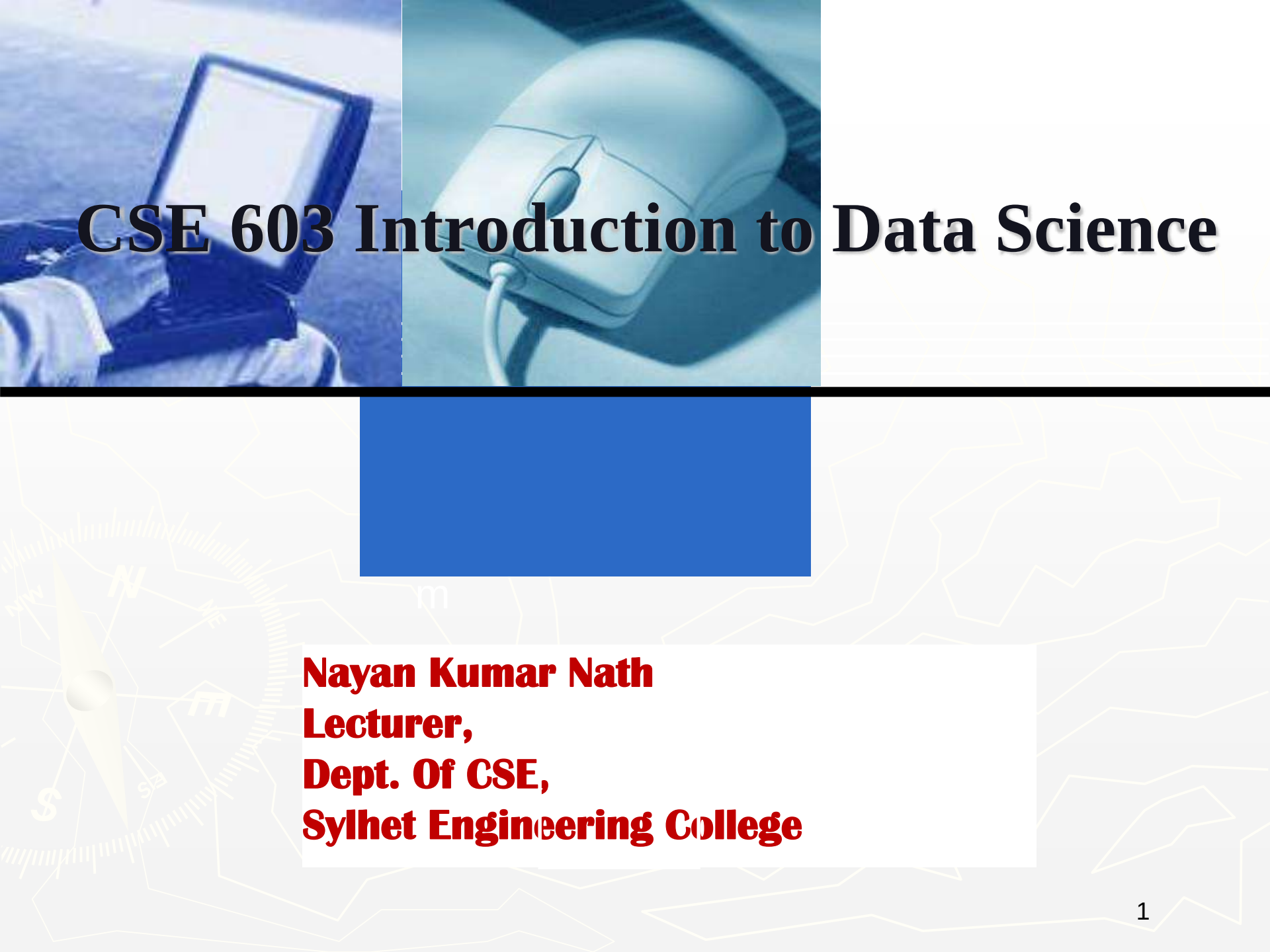




CSE 603 Introduction to Data Science

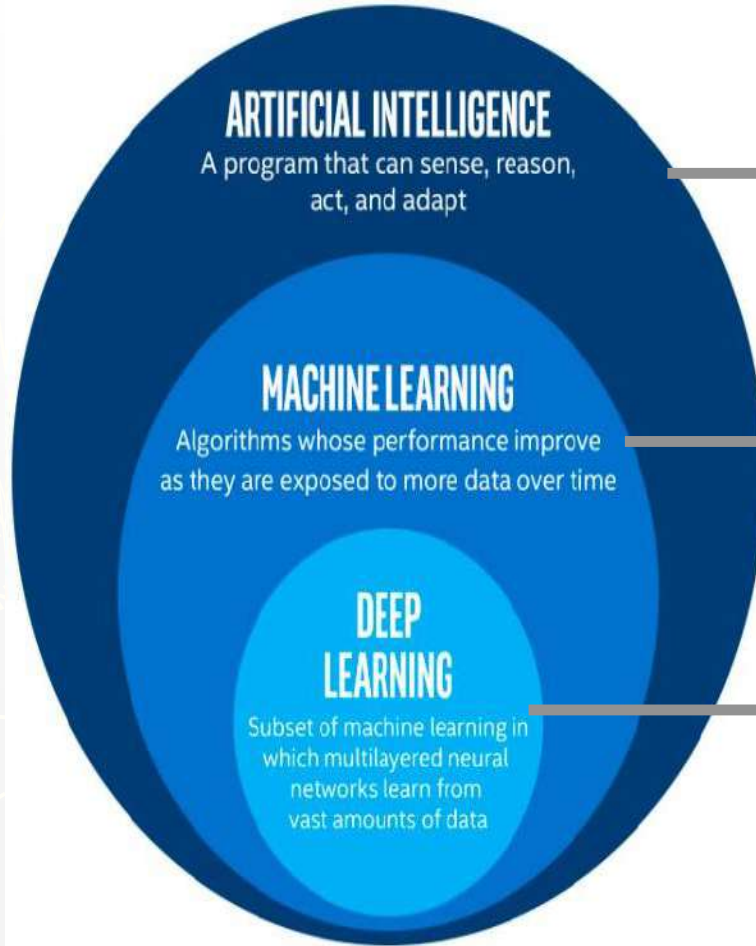
Nayan Kumar Nath
Lecturer,
Dept. Of CSE,
Sylhet Engineering College

What is machine learning?



Blue faces seem to be important...

What is machine learning really?

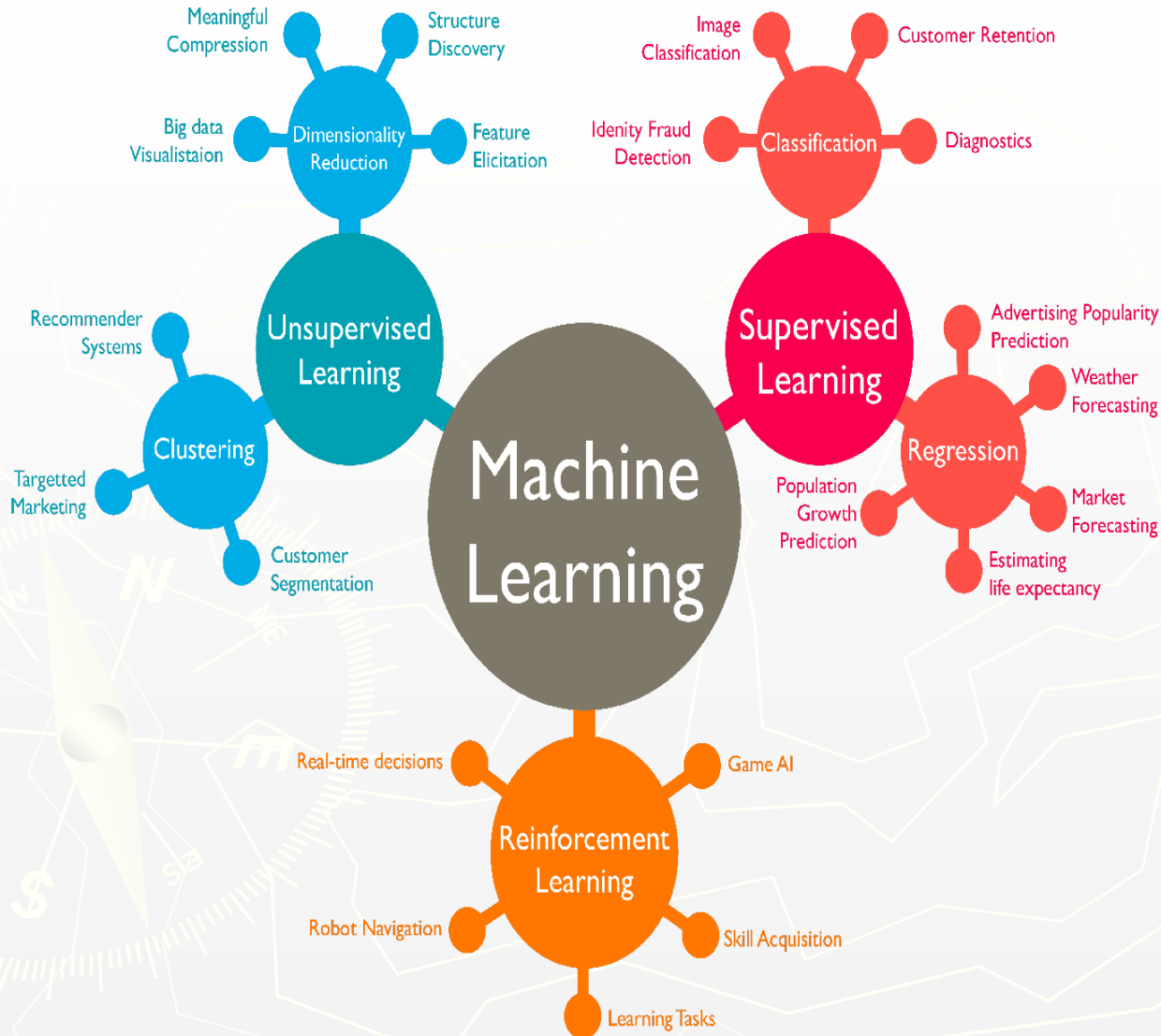


1950's: creation of first “intelligent” algorithms and programs. Alan Turing published “Computer Machinery and Intelligence” which proposed a test of machine intelligence called The Imitation Game

1980's: statistical models and algorithms that can learn from data

2010's: statistical models and algorithms inspired by neurones that can learn from data

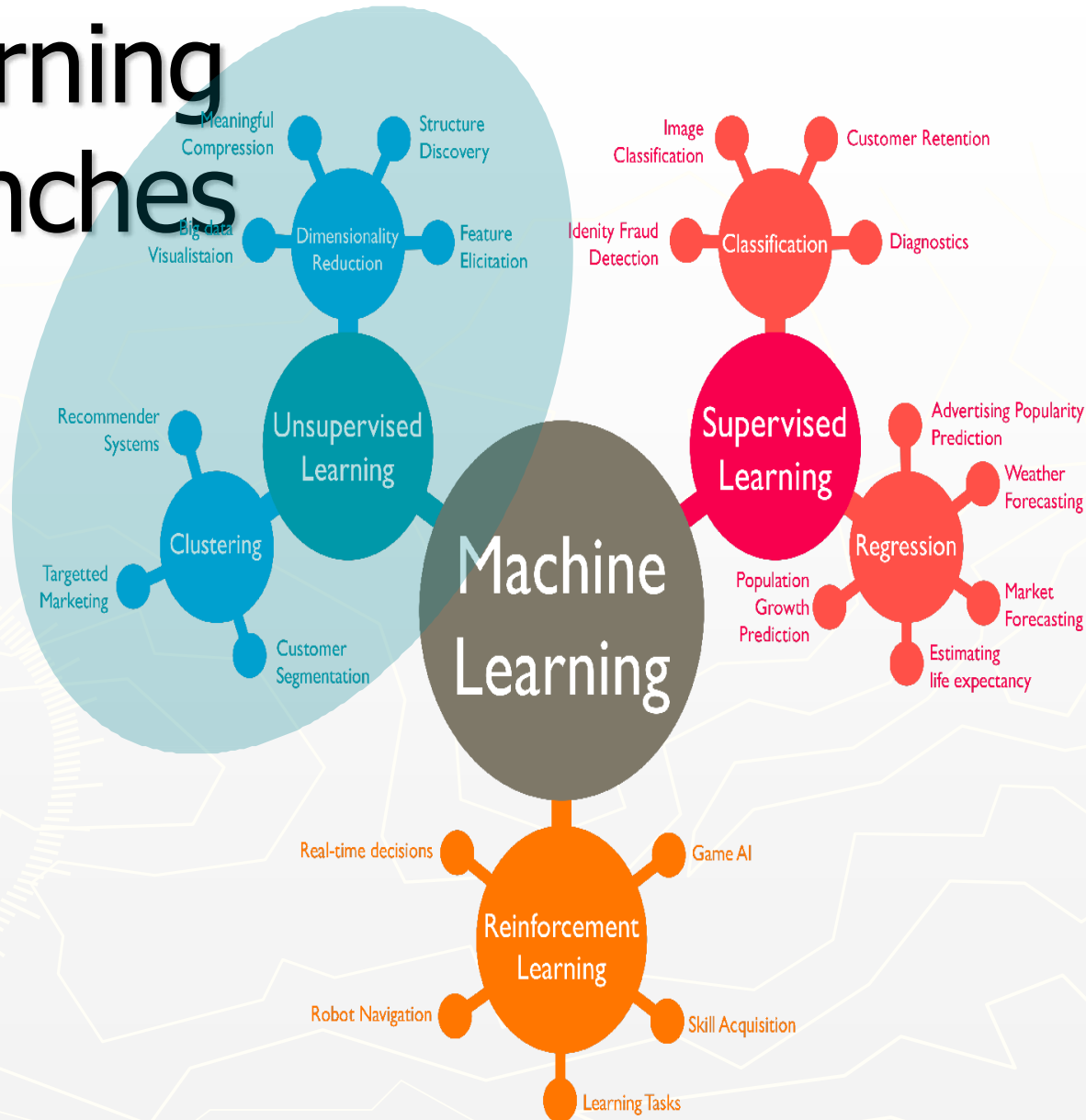
Machine Learning Branches



3 Main Branches:

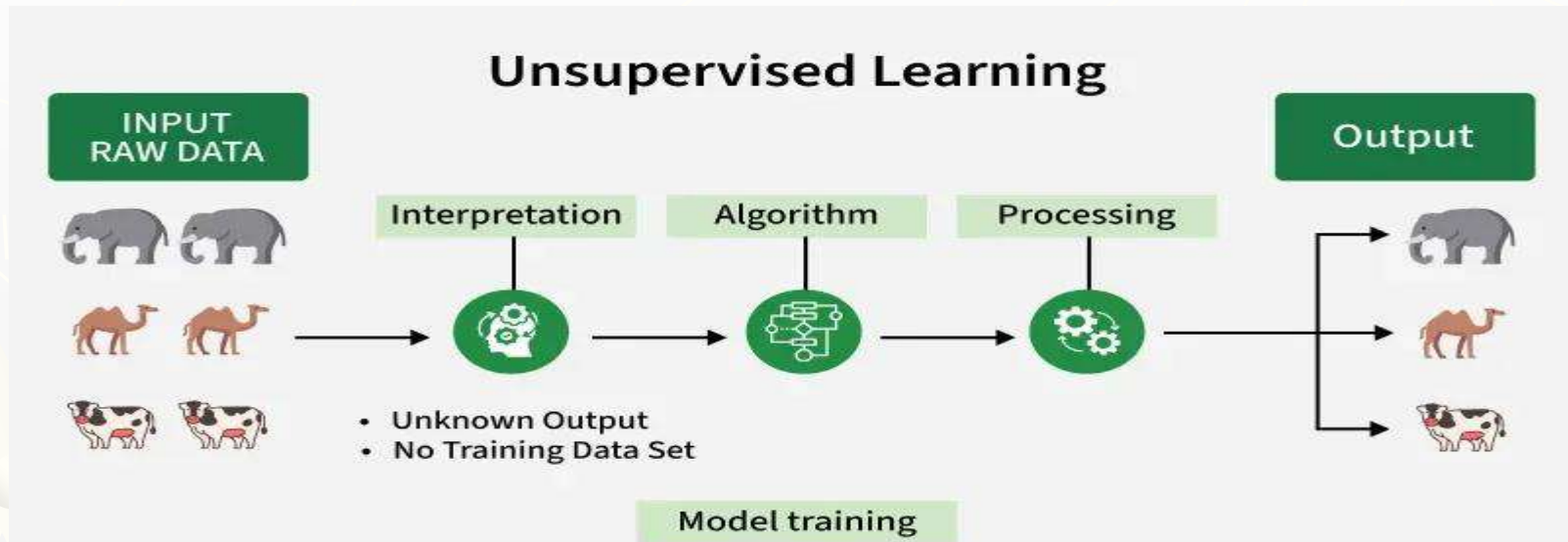
- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning

Machine Learning Branches



Machine Learning Branches: Unsupervised Learning

In **unsupervised learning** there are no labels available, insights are gained without* prior knowledge. The main aim of the unsupervised learning algorithm is to group or categories the unsorted dataset according to the similarities, patterns, and differences. It learns patterns on its own by grouping similar data points or finding hidden structures without any human intervention.



The image shows set of animals like elephants, camels and cows that represents raw data that the unsupervised learning algorithm will process.

The output shows the results of the unsupervised learning process. In this case, the algorithm might have grouped the animals into clusters based on their species (elephants, camels, cows).

Working of Unsupervised Learning

The working of unsupervised machine learning can be explained in these steps:

1. Collect Unlabeled Data

- Gather a dataset without predefined labels or categories.

- Example:** Images of various animals without any tags.

2. Select an Algorithm

- Choose a suitable unsupervised algorithm such as clustering like K-Means, association rule learning like Apriori or dimensionality reduction like PCA based on the goal.

3. Train the Model on Raw Data

- Feed the entire unlabeled dataset to the algorithm.

- The algorithm looks for similarities, relationships or hidden structures within the data.

4. Group or Transform Data

- The algorithm organizes data into groups (clusters), rules or lower-dimensional forms without human input.

- Example: It may group similar animals together or extract key patterns from large datasets.

5. Interpret and Use Results

- Analyze the discovered groups, rules or features to gain insights or use them for further tasks like visualization, anomaly detection or as input for other models.

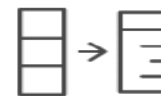
Types of Unsupervised Learning



Clustering



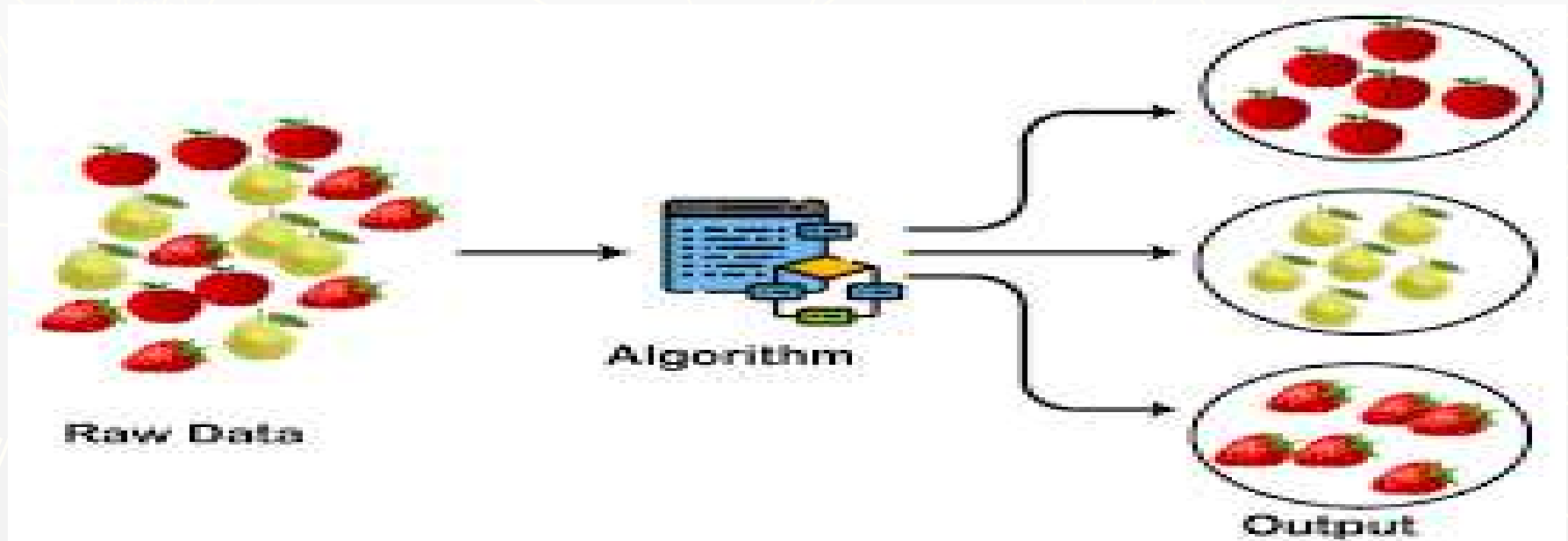
Association Rule
Learning



Dimensionality
Reduction

Unsupervised Learning: Clustering

Clustering: The clustering technique is used when we want to find the inherent groups from the data. It is a way to group the objects into a cluster such that the objects with the most similarities remain in one group and have fewer or no similarities with the objects of other groups. An example of the clustering algorithm is grouping the customers by their purchasing behaviour. Some of the popular clustering algorithms are given below: K-Means Clustering algorithm, Hierarchical Clustering, Mean-shift algorithm, DBSCAN (Density-based spatial clustering of applications with noise) Algorithm, Spectral Clustering



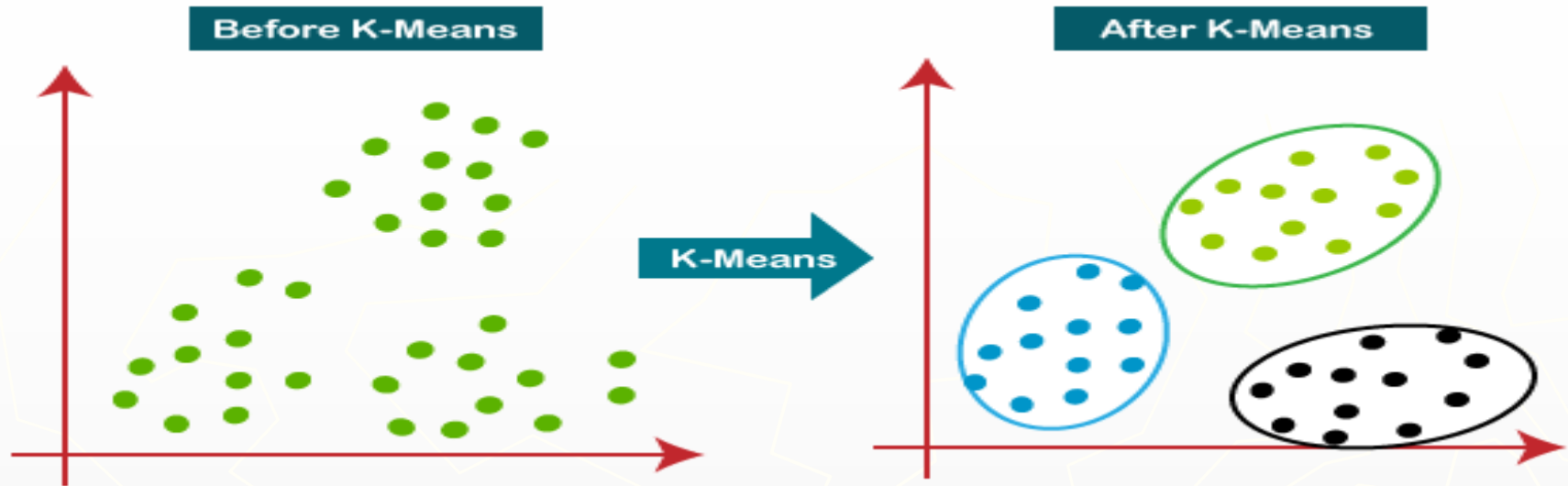
K-Means Clustering Algorithm

- K-Means Clustering is an Unsupervised Learning algorithm which groups the unlabeled dataset into different clusters. Here K defines the number of pre-defined clusters that need to be created in the process, as if $K=2$, there will be two clusters, and for $K=3$, there will be three clusters, and so on. It is an iterative algorithm that divides the unlabeled dataset into k different clusters in such a way that each dataset belongs only one group that has similar properties.
- It allows us to cluster the data into different groups and a convenient way to discover the categories of groups in the unlabeled dataset on its own without the need for any training.
- It is a centroid-based algorithm, where each cluster is associated with a centroid. The main aim of this algorithm is to minimize the sum of distances between the data point and their corresponding clusters.
- The algorithm takes the unlabeled dataset as input, divides the dataset into k-number of clusters, and repeats the process until it does not find the best clusters. The value of k should be predetermined in this algorithm.
- The k-means clustering algorithm mainly performs two tasks:
 - Determines the best value for K center points or centroids by an iterative process.
 - Assigns each data point to its closest k-center. Those data points which are near to the particular k-center, create a cluster.

Hence each cluster has datapoints with some commonalities, and it is away from other clusters.

K-Means Clustering Algorithm

- The below diagram explains the working of the K-means Clustering Algorithm:



How does the K-Means Algorithm Work?

The working of the K-Means algorithm is explained in the below steps:

Step-1: Select the number K to decide the number of clusters.

Step-2: Select random K points or centroids. (It can be other from the input dataset).

Step-3: Assign each data point to their closest centroid, which will form the predefined K clusters.

Step-4: Calculate the variance and place a new centroid of each cluster.

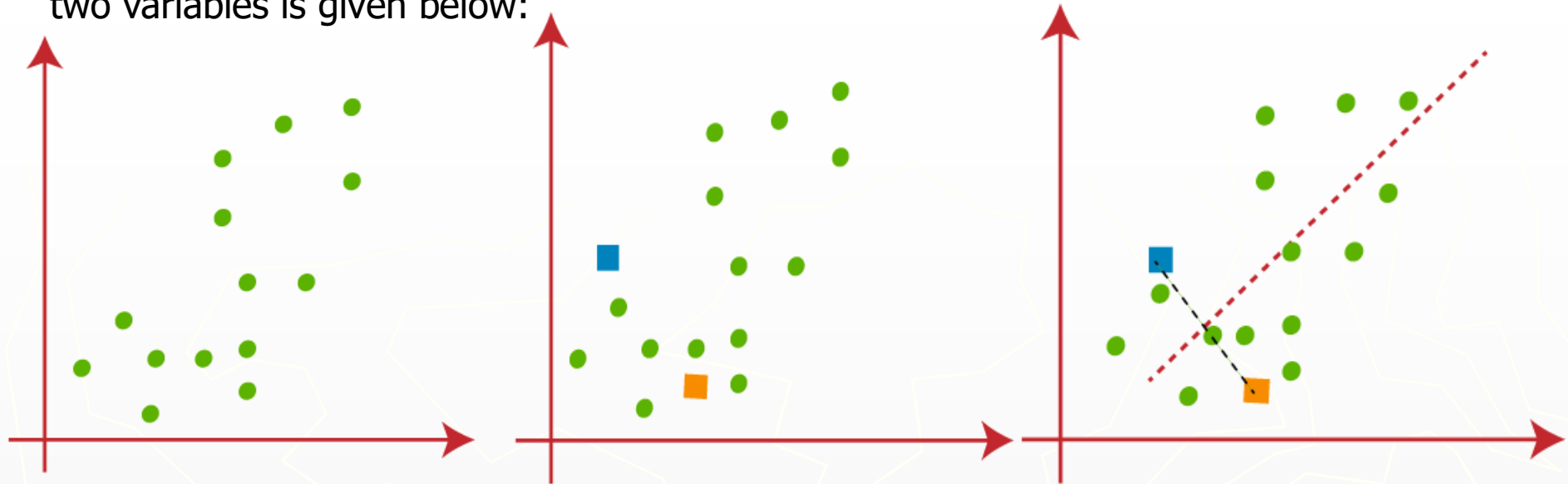
Step-5: Repeat the third steps, which means reassign each datapoint to the new closest centroid of each cluster.

Step-6: If any reassignment occurs, then go to step-4 else go to FINISH.

Step-7: The model is ready.

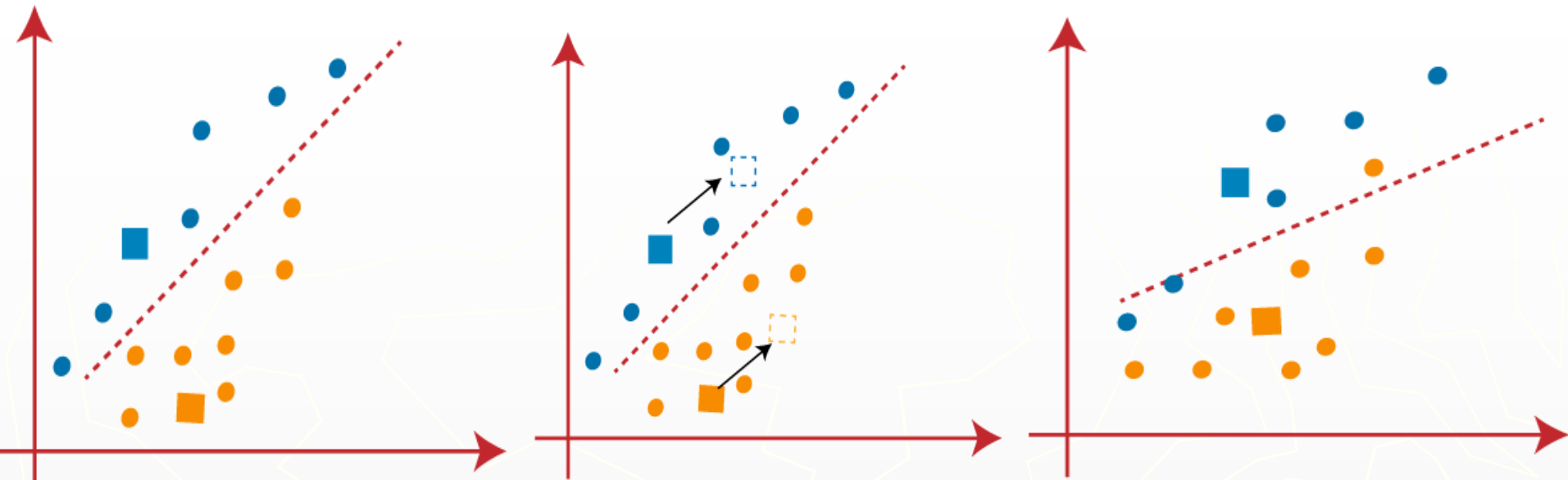
K-Means Clustering Algorithm

- Suppose we have two variables M1 and M2. The x-y axis scatter plot((fig.1)) of these two variables is given below:



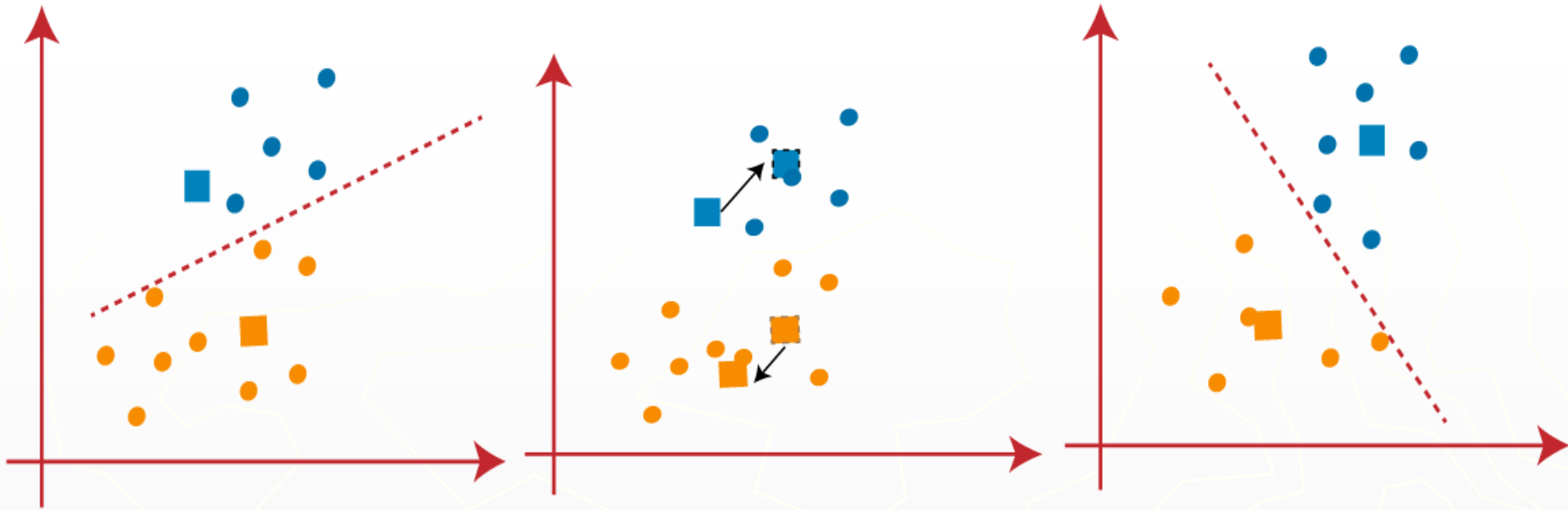
- Let's take number k of clusters, i.e., $K=2$, to identify the dataset and to put them into different clusters. It means here we will try to group these datasets into two different clusters.
- We need to choose some random k points or centroid to form the cluster. These points can be either the points from the dataset or any other point. So, here we are selecting the below two points as k points, which are not the part of our dataset. Consider the above image(fig 2):
- Now we will assign each data point of the scatter plot to its closest K -point or centroid. We will compute it by applying some mathematics that we have studied to calculate the distance between two points. So, we will draw a median between both the centroids. Consider the above image(fig. 3).
- From the above image, it is clear that points left side of the line is near to the $K1$ or blue centroid, and points to the right of the line are close to the yellow centroid. Let's color them as blue and yellow for clear visualization.

K-Means Clustering Algorithm



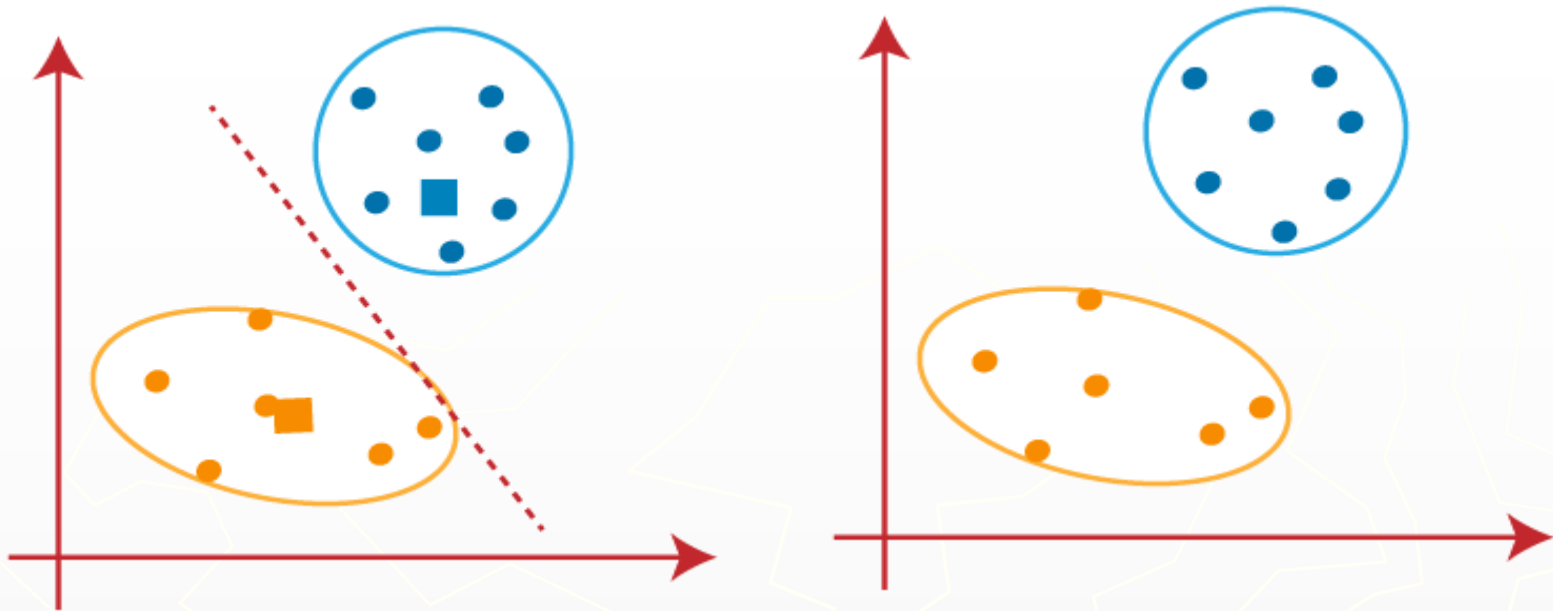
- As we need to find the closest cluster, so we will repeat the process by choosing a **new centroid**. To choose the new centroids, we will compute the center of gravity of these centroids, and will find new centroids (fig 1.)
- Next, we will reassign each data point to the new centroid. For this, we will repeat the same process of finding a median line. The median will be like next image(fig. 2)
- From the above image, we can see, one yellow point is on the left side of the line, and two blue points are right to the line. So, these three points will be assigned to new centroids.(fig. 3)

K-Means Clustering Algorithm



- As reassignment has taken place, so we will again go to the step-4, which is finding new centroids or K-points.
- We will repeat the process by finding the center of gravity of centroids, so the new centroids will be as shown in the below image:(fig 1.)
- As we got the new centroids so again will draw the median line and reassign the data points. So, the image will be(fig. 2):

K-Means Clustering Algorithm



•We can see in the above image; there are no dissimilar data points on either side of the line, which means our model is formed. Consider the below image.(fig. 1)

•As our model is ready, so we can now remove the assumed centroids, and the two final clusters will be as shown in the below image(fig. 2)

•How to choose the value of "K number of clusters" in K-means Clustering?

The performance of the K-means clustering algorithm depends upon highly efficient clusters that it forms. But choosing the optimal number of clusters is a big task. There are some different ways to find the optimal number of clusters, but here we are discussing the most appropriate method to find the number of clusters or value of K. The method is given next slide:

K-Means Clustering Algorithm

Elbow Method

•The Elbow method is one of the most popular ways to find the optimal number of clusters. This method uses the concept of WCSS value. **WCSS** stands for **Within Cluster Sum of Squares**, which defines the total variations within a cluster. The formula to calculate the value of WCSS (for 3 clusters) is given below:

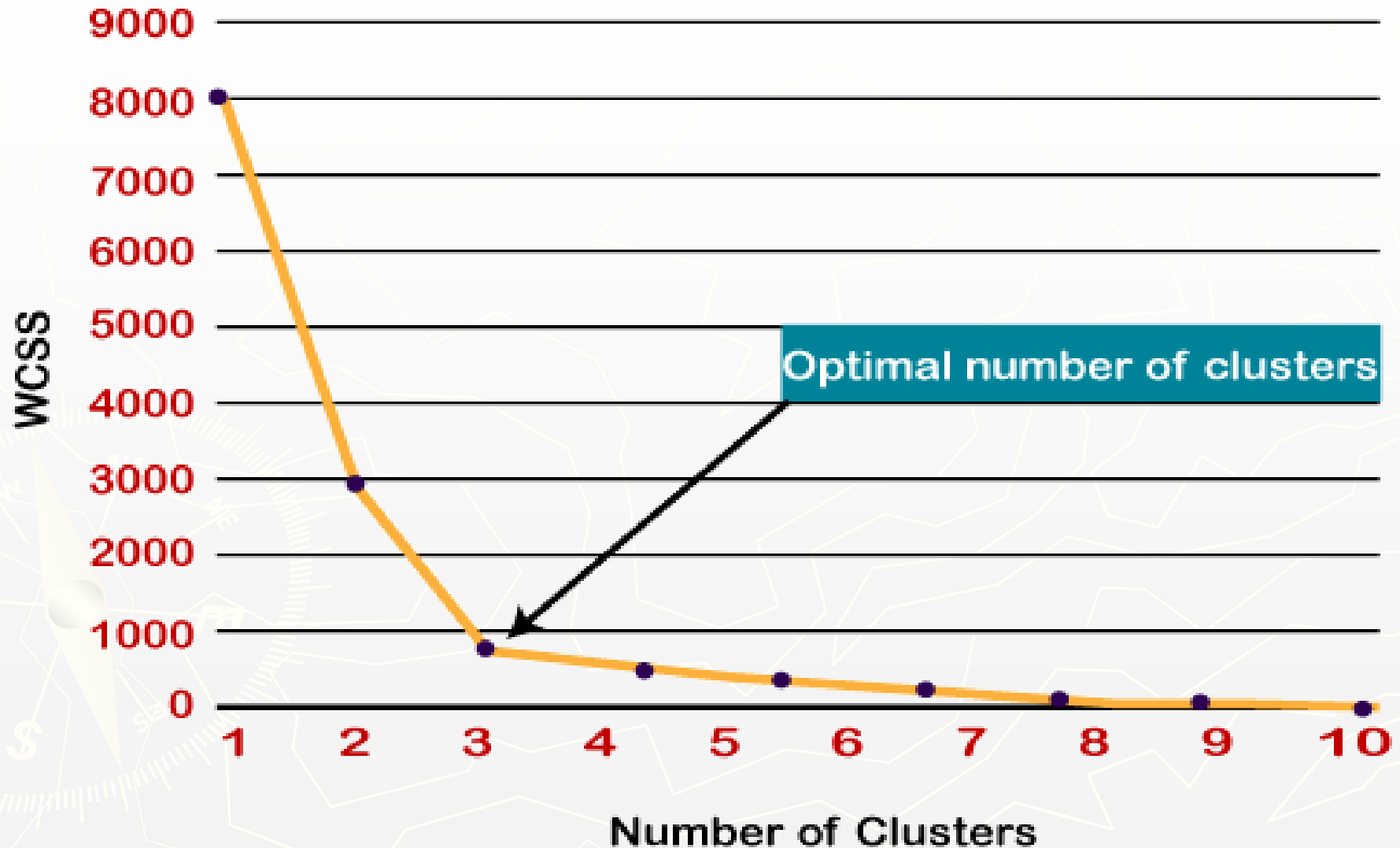
$$\bullet \text{WCSS} = \sum_{P_i \text{ in Cluster1}} \text{distance}(P_i, C_1)^2 + \sum_{P_i \text{ in Cluster2}} \text{distance}(P_i, C_2)^2 + \sum_{P_i \text{ in Cluster3}} \text{distance}(P_i, C_3)^2$$

In the above formula of WCSS,

- $\sum_{P_i \text{ in Cluster1}} \text{distance}(P_i, C_1)^2$: It is the sum of the square of the distances between each data point and its centroid within a cluster1 and the same for the other two terms.
- To measure the distance between data points and centroid, we can use any method such as Euclidean distance or Manhattan distance.
- To find the optimal value of clusters, the elbow method follows the below steps:
- It executes the K-means clustering on a given dataset for different K values (ranges from 1-10).
- For each value of K, calculates the WCSS value.
- Plots a curve between calculated WCSS values and the number of clusters K.
- The sharp point of bend or a point of the plot looks like an arm, then that point is considered as the best value of K.

K-Means Clustering Algorithm: Example

Since the graph shows the sharp bend, which looks like an elbow, hence it is known as the elbow method. The graph for the elbow method looks like the below image:



Distance Measures

The following are the different Distance measures:

a. Euclidean Distance :

$$D_e(p, q) = [(x-s)^2 + (y-t)^2]^{1/2}$$

b. City Block Distance:

$$D_4(p, q) = |x-s| + |y-t|$$

c. Chess Board Distance:

$$D_8(p, q) = \max(|x-s|, |y-t|)$$

		2		
	2	1	2	
2	1	0	1	2
	2	1	2	
		2		

2	2	2	2	2
2	1	1	1	2
2	1	0	1	2
2	1	1	1	2
2	2	2	2	2

Distance Measures

Euclidean Distance:

The *Euclidean distance* between p and q is given by:

$$D_e(p, q) = [(x-s)^2 + (y-t)^2]^{1/2}$$

The pixels having distance less than or equal to some value r from (x,y) are the points contained in a disk of radius r centered at (x,y)

City Block Distance:

$$D_4(p, q) = |x-s| + |y-t|$$

- The pixels having a D_4 distance less than some r from (x,y) form a diamond centered at (x,y)
- Example: pixels where $D_4 \leq 2$

		2		
	2	1	2	
2	1	0	1	2
	2	1	2	
		2		

[Note: Pixels with $D_4=1$ are the 4-neighbors of (x,y)]

Distance Measures

Chess Board Distance:

The D_8 distance (also called the *chessboard distance*) between p and q is given by:

$$D_8(p, q) = \max(|x-s|, |y-t|)$$

- The pixels having a D_8 distance less than some r from (x,y) form a square centered at (x,y)
- Example: pixels where $D_8 \leq 2$

[Note: Pixels with $D_8=1$ are the 8-neighbors of (x,y)]



K-Means Clustering Algorithm-Solved example

Given Data Points:

We have the following 5 points in 2D space:

$$(2, 3), (5, 8), (9, 10), (4, 2), (6, 9)$$

We will cluster them into $K = 2$ clusters.

Step 1: Initialize Centroids

Let's randomly pick two points as initial centroids:

$$C_1 = (2, 3), \quad C_2 = (9, 10)$$

Step 2: Compute Distance from Each Point to the Centroids

We use Euclidean distance formula:

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Means Clustering Algorithm-Solved example

Distances to $C_1 = (2, 3)$ and $C_2 = (9, 10)$:

Point (x, y)	Distance to $C_1 = (2, 3)$	Distance to $C_2 = (9, 10)$	Assigned Cluster
(2,3)	0.00	9.90	Cluster 1
(5,8)	5.83	5.00	Cluster 2
(9,10)	9.90	0.00	Cluster 2
(4,2)	2.24	9.22	Cluster 1
(6,9)	7.81	3.61	Cluster 2

After the first iteration, we have:

- Cluster 1: (2,3), (4,2)
- Cluster 2: (5,8), (9,10), (6,9)



Step 3: Compute New Centroids

New centroid is the **mean** of all points in a cluster.

New Centroids:

$$C_1 = \left(\frac{2 + 4}{2}, \frac{3 + 2}{2} \right) = (3, 2.5)$$
$$C_2 = \left(\frac{5 + 9 + 6}{3}, \frac{8 + 10 + 9}{3} \right) = (6.67, 9)$$

K-Means Clustering Algorithm-Solved example

Step 4: Recompute Distances & Assign Points Again

We recalculate distances and reassign points.

Point (x, y)	Distance to $C_1 = (3, 2.5)$	Distance to $C_2 = (6.67, 9)$	Assigned Cluster
(2,3)	1.12	8.31	Cluster 1
(5,8)	5.70	1.94	Cluster 2
(9,10)	9.04	2.65	Cluster 2
(4,2)	1.12	7.43	Cluster 1
(6,9)	7.05	0.67	Cluster 2

No changes in cluster assignments, so the algorithm converges!

Final Clusters:

- **Cluster 1:** (2,3), (4,2) → Centroid (3,2.5)
- **Cluster 2:** (5,8), (9,10), (6,9) → Centroid (6.67,9)

This is how K-Means clustering works mathematically!

K-Means Clustering – Solved Example

- Suppose that the data mining task is to cluster points into three clusters,
- where the points are
- $A1(2, 10)$, $A2(2, 5)$, $A3(8, 4)$, $B1(5, 8)$, $B2(7, 5)$, $B3(6, 4)$, $C1(1, 2)$, $C2(4, 9)$.
- The distance function is Euclidean distance.
- Suppose initially we assign $A1$, $B1$, and $C1$ as the center of each cluster, respectively.

K-Means Clustering – Solved Example

Initial Centroids:

A1: (2, 10)

B1: (5, 8)

C1: (1, 2)

New Centroids:

A1: (2, 10) ✓

B1: (6, 6) ✓

C1: (1.5, 3.5) ✓

Data Points			Distance to						Cluster	New Cluster
			2	10	5	8	1	2		
A1	2	10	0.00		3.61		8.06		1	
A2	2	5	5.00		4.24		3.16		3	
A3	8	4	8.49		5.00		7.28		2	
B1	5	8	3.61		0.00		7.21		2	
B2	7	5	7.07		3.61		6.71		2	
B3	6	4	7.21		4.12		5.39		2	
C1	1	2	8.06		7.21		0.00		3	
C2	4	9	2.24		1.41		7.62		2	

Assign cluster which one has smallest distance to initial centroid

$$d(p_1, p_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Means Clustering – Solved Example

Current Centroids:

A1: (2, 10)

B1: (6, 6)

C1: (1.5, 3.5)

Data Points			Distance to						Cluster	New Cluster
			2	10	6	6	1.5	1.5		
A1	2	10	0.00		5.66		6.52		1	1
A2	2	5	5.00		4.12		1.58		3	3
A3	8	4	8.49		2.83		6.52		2	2
B1	5	8	3.61		2.24		5.70		2	2
B2	7	5	7.07		1.41		5.70		2	2
B3	6	4	7.21		2.00		4.53		2	2
C1	1	2	8.06		6.40		1.58		3	3
C2	4	9	2.24		3.61		6.04		2	1

$$d(p_1, p_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Means Clustering – Solved Example

Current Centroids:

A1: (2, 10)

B1: (6, 6)

C1: (1.5, 3.5)

New Centroids:

A1: (3, 9.5) ✓

B1: (6.5, 5.25) ✓

C1: (1.5, 3.5) ✓

Data Points			Distance to						Cluster	New Cluster
			2	10	6	6	1.5	1.5		
A1	2	10	0.00		5.66		6.52		1	1
A2	2	5	5.00		4.12		1.58		3	3
A3	8	4	8.49		2.83		6.52		2	2
B1	5	8	3.61		2.24		5.70		2	2
B2	7	5	7.07		1.41		5.70		2	2
B3	6	4	7.21		2.00		4.53		2	2
C1	1	2	8.06		6.40		1.58		3	3
C2	4	9	2.24		3.61		6.04		2	1

$$d(p_1, p_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Means Clustering – Solved Example

Current Centroids:

A1: (3, 9.5)

B1: (6.5, 5.25)

C1: (1.5, 3.5)

New Centroids:

A1: (3.67, 9)

B1: (7, 4.33)

C1: (1.5, 3.5)

Data Points			Distance to						Cluster	New Cluster
			3	9.5	6.5	5.25	1.5	3.5		
A1	2	10	1.12		6.54		6.52		1	
A2	2	5	4.61		4.51		1.58		3	
A3	8	4	7.43		1.95		6.52		2	
B1	5	8	2.50		3.13		5.70		1	
B2	7	5	6.02		0.56		5.70		2	
B3	6	4	6.26		1.35		4.53		2	
C1	1	2	7.76		6.39		1.58		3	
C2	4	9	1.12		4.51		6.04		1	

$$d(p_1, p_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Means Clustering – Solved Example

Current Centroids:

A1: (3.67, 9)

B1: (7, 4.33)

C1: (1.5, 3.5)

Data Points			Distance to						Cluster	New Cluster
			3.67	9	7	4.33	1.5	3.5		
A1	2	10	1.94		7.56		6.52		1	1
A2	2	5	4.33		5.04		1.58		3	3
A3	8	4	6.62		1.05		6.52		2	2
B1	5	8	1.67		4.18		5.70		1	1
B2	7	5	5.21		0.67		5.70		2	2
B3	6	4	5.52		1.05		4.53		2	2
C1	1	2	7.49		6.44		1.58		3	3
C2	4	9	0.33		5.55		6.04		1	1

$$d(p_1, p_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Advantages and Disadvantages of K-Means Clustering

▪ Advantages of K-Means Clustering

1. Simple and Easy to Understand
2. Fast and Efficient: K-Means works efficiently on large datasets.
3. Works Well with Numerical Data
4. Easy Visualization
5. Scalable
6. Produces Compact Clusters
7. Widely Used in Real Applications

▪ Disadvantages of K-Means Clustering

1. Need to Choose K Manually
2. Sensitive to Initial Centroids
3. Sensitive to Outliers
4. Cannot Handle Non-Spherical Cluster
5. Works Poorly with Different Cluster Sizes
6. Only Works with Numerical Data
7. Local Optimum Problem

Hierarchical clustering

- Hierarchical clustering refers to an unsupervised learning procedure that determines successive clusters based on previously defined clusters. It works via grouping data into a tree of clusters. Hierarchical clustering starts by treating each data point as an individual cluster. The endpoint refers to a different set of clusters, where each cluster is different from the other cluster, and the objects within each cluster are the same as one another.

There are two types of hierarchical clustering

- Agglomerative Hierarchical Clustering
- Divisive Clustering

Agglomerative hierarchical clustering

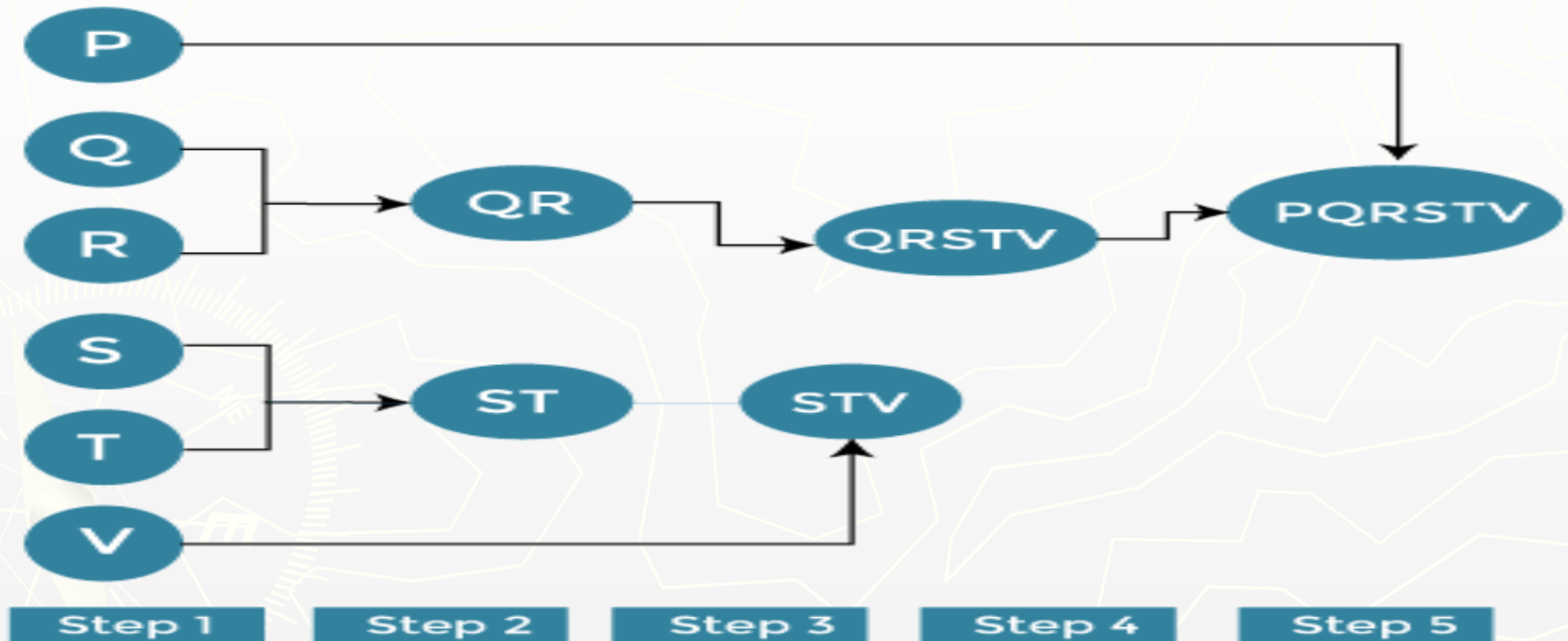
Agglomerative clustering is one of the most common types of hierarchical clustering used to group similar objects in clusters. Agglomerative clustering is also known as AGNES (Agglomerative Nesting). In agglomerative clustering, each data point acts as an individual cluster and at each step, data objects are grouped in a bottom-up method. Initially, each data object is in its cluster. At each iteration, the clusters are combined with different clusters until one cluster is formed.

Agglomerative hierarchical clustering algorithm

- Determine the similarity between individuals and all other clusters. (Find proximity matrix).
- Consider each data point as an individual cluster.
- Combine similar clusters.
- Recalculate the proximity matrix for each cluster.
- Repeat step 3 and step 4 until you get a single cluster.

Hierarchical clustering

- Let's understand this concept with the help of graphical representation using a dendrogram. [A dendrogram is a branching diagram that represents the relationships of similarity among a group of entities]
- With the help of given demonstration, we can understand that how the actual algorithm work. Here no calculation has been done below all the proximity among the clusters are assumed.
- Let's suppose we have six different data points P, Q, R, S, T, V.



Step 1:

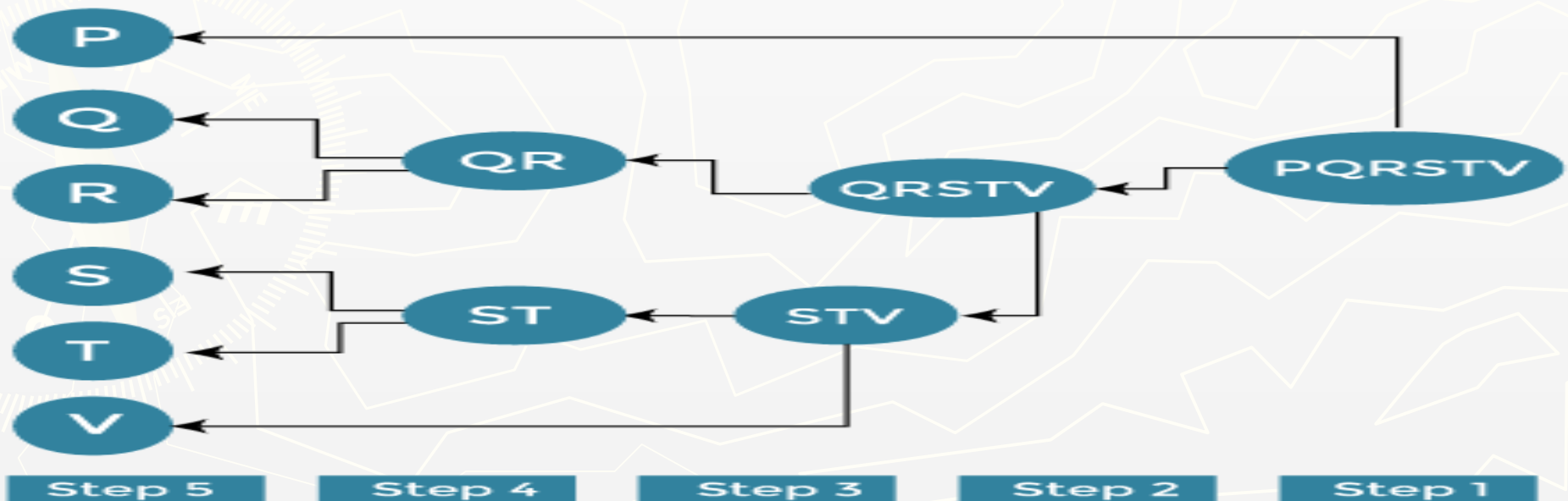
Consider each alphabet (P, Q, R, S, T, V) as an individual cluster and find the distance between the individual cluster from all other clusters.

Hierarchical clustering

- **Step 2:** Now, merge the comparable clusters in a single cluster. Let's say cluster Q and Cluster R are similar to each other so that we can merge them in the second step. Finally, we get the clusters $[(P), (QR), (ST), (V)]$
- **Step 3:** Here, we recalculate the proximity as per the algorithm and combine the two closest clusters $[(ST), (V)]$ together to form new clusters as $[(P), (QR), (STV)]$
- **Step 4:** Repeat the same process. The clusters STV and PQ are comparable and combined together to form a new cluster. Now we have $[(P), (QQRSTV)]$.
- **Step 5:** Finally, the remaining two clusters are merged together to form a single cluster $[(PQRSTV)]$

Divisive Hierarchical Clustering

Divisive hierarchical clustering is exactly the opposite of Agglomerative Hierarchical clustering. In Divisive Hierarchical clustering, all the data points are considered an individual cluster, and in every iteration, the data points that are not similar are separated from the cluster. The separated data points are treated as an individual cluster. Finally, we are left with N clusters.



Hierarchical clustering

Agglomerative Hierarchical Clustering Solved Example

- Consider the following set of 6 one dimensional data points:
- 18, 22, 25, 42, 27, 43
- Apply the **agglomerative hierarchical clustering** algorithm to build the hierarchical clustering **dendrogram**.
- Merge the clusters using **Min distance** and update the proximity matrix accordingly.
- Clearly show the **proximity matrix** corresponding to each iteration of the algorithm.

Step 1: Initial Proximity Matrix

Compute pairwise distances.

	18	22	25	27	42	43
18	0	4	7	9	24	25
22	4	0	3	5	20	21
25	7	3	0	2	17	18
27	9	5	2	0	15	16
42	24	20	17	15	0	1
43	25	21	18	16	1	0

Agglomerative Hierarchical Clustering Solved Example

Step 2: Find Minimum Distance

- Step – 1

	18	22	25	27	42	43
18	0	4	7	9	24	25
22	4	0	3	5	20	21
25	7	3	0	2	17	18
27	9	5	2	0	15	16
42	24	20	17	15	0	1
43	25	21	18	16	1	0

- The matrix is **symmetric**:
 $d(a,b)=d(b,a)$
Smallest distance is:
 $(42,43)=1$

So the first merge is: **(42, 43)** **Clusters now:** 18, 22, 25, 27, (42,43)

Step 3: Update Matrix

- Step – 2

Using Single Linkage: $d((42,43),x)=\min(d(42,x),d(43,x))$

	18	22	25	27	42, 43
18	0	4	7	9	24
22	4	0	3	5	20
25	7	3	0	2	17
27	9	5	2	0	15
42, 43	24	20	17	15	0

Agglomerative Hierarchical Clustering Solved Example

Step 4: Find Next Minimum Distance

Smallest distance is: $(25, 27) = 2$

- Step – 2

	18	22	25	27	42, 43
18	0	4	7	9	24
22	4	0	3	5	20
25	7	3	0	2	17
27	9	5	2	0	15
42, 43	24	20	17	15	0

$(42, 43), (25, 27)$ Clusters now:
18, 22, $(25, 27)$, $(42, 43)$

- Step – 3

	18	22	25, 27	42, 43
18	0	4	7	24
22	4	0	3	20
25, 27	7	3	0	15
42, 43	24	20	15	0

Agglomerative Hierarchical Clustering Solved Example

- Step – 3

	18	22	25, 27	42, 43
18	0	4	7	24
22	4	0	3	20
25, 27	7	3	0	15
42, 43	24	20	15	0

(42, 43), ((25, 27), 22)

- Step – 4

	18	22, 25, 27	42, 43
18	0	4	24
22, 25, 27	4	0	15
42, 43	24	15	0

Agglomerative Hierarchical Clustering Solved Example

- Step – 4

	18	22, 25, 27	42, 43
18	0	4	24
22, 25, 27	4	0	15
42, 43	24	15	0

$(42, 43), ((25, 27), 22), 18)$

- Step – 5

	18, 22, 25, 27	42, 43
18, 22, 25, 27	0	15
42, 43	15	0

	18, 22, 25, 27	42, 43
18, 22, 25, 27	0	15
42, 43	15	0

$((42, 43), ((25, 27), 22), 18)$

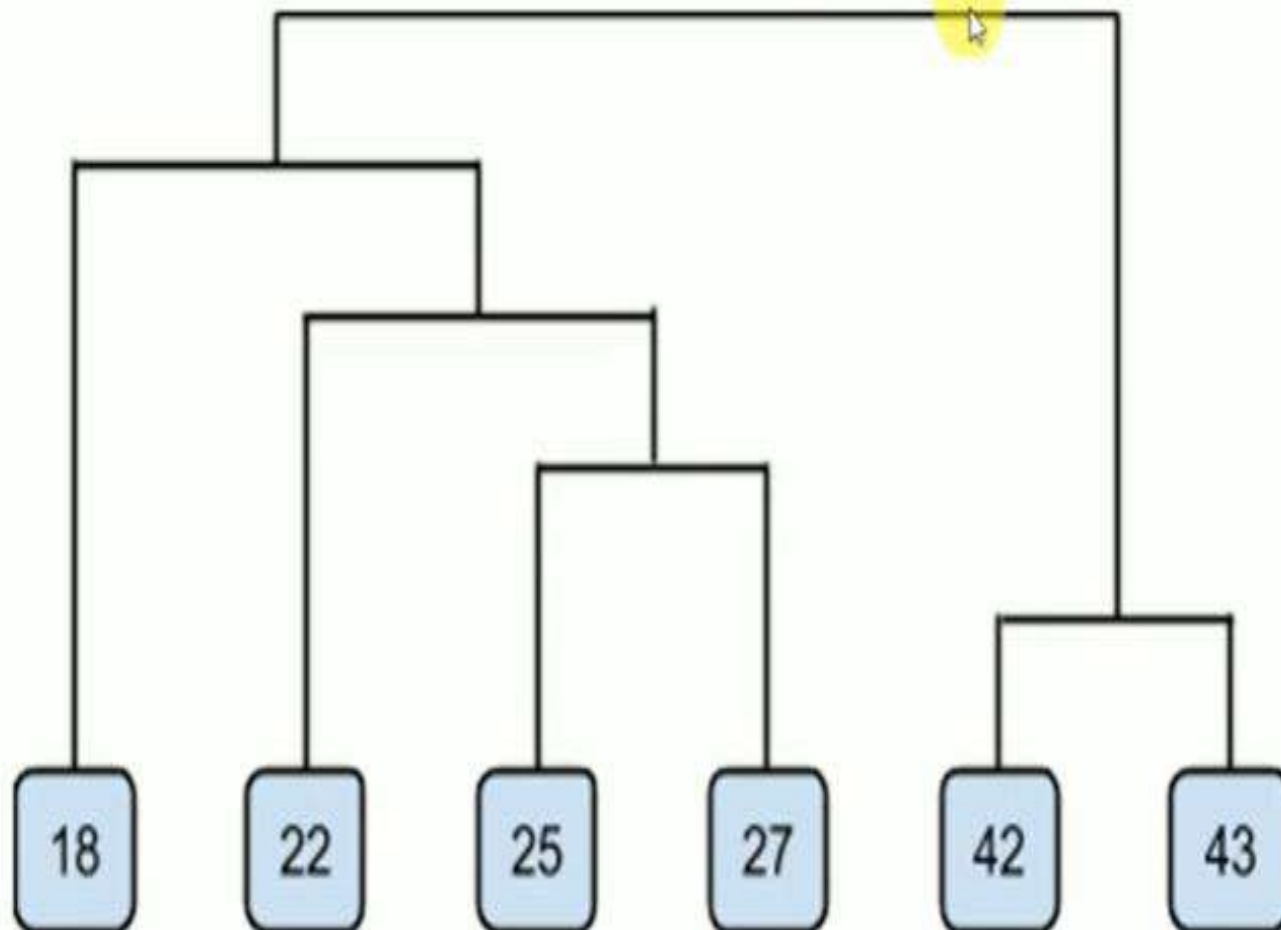
- Step – 6

	18, 22, 25, 27, 42, 43
18, 22, 25, 27, 42, 43	0

Agglomerative Hierarchical Clustering Solved Example

- Dendrogram

$((42, 43), ((25, 27), 22), 18)$



Hierarchical clustering

Advantages of Hierarchical clustering

- It is simple to implement and gives the best output in some cases.
- It is easy and results in a hierarchy, a structure that contains more information.
- It does not need us to pre-specify the number of clusters.

Disadvantages of hierarchical clustering

- It breaks the large clusters.
- It is Difficult to handle different sized clusters and convex shapes.
- It is sensitive to noise and outliers.
- The algorithm can never be changed or deleted once it was done previously.

Among these linkage methods, we have the following ones:

"single": the distance between clusters equals the minimum distance between individuals.

"complete": the distance between clusters equals the maximum distance between individuals.

"arithmetic": the distance between clusters equals the arithmetic mean distance between individuals. Also known as average linkage, WPGMA (weighted version) or UPGMA (unweighted version).

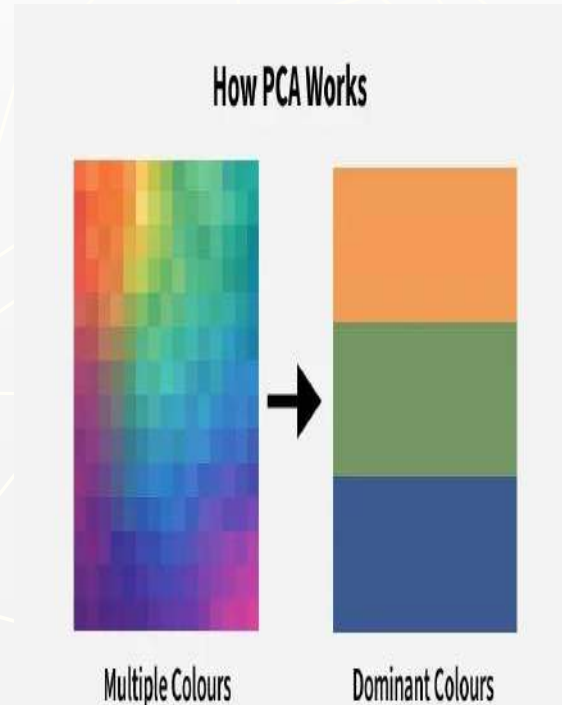
"geometric": the distance between clusters equals the geometric mean distance between individuals.

"harmonic": the distance between clusters equals the harmonic mean distance between individuals.

"versatile": the distance between clusters equals the generalized power mean distance between individuals.

Principal Component Analysis Algorithm

- Principal Component Analysis is an unsupervised learning algorithm that is used for the dimensionality reduction in machine learning and helps us to reduce the number of features in a dataset while keeping the most important information. It is a statistical process that converts the observations of correlated features into a set of linearly uncorrelated features with the help of orthogonal transformation. These new transformed features are called the **Principal Components**. It changes complex datasets by transforming correlated features into a smaller set of uncorrelated components. It is a technique to draw strong patterns from the given dataset by reducing the variances. PCA generally tries to find the lower-dimensional surface to project the high-dimensional data.
- PCA works by considering the variance of each attribute because the high attribute shows the good split between the classes, and hence it reduces the dimensionality. Some real-world applications of PCA are **image processing, movie recommendation system, optimizing the power allocation** in various communication channels. It is a feature extraction technique, so it contains the important variables and drops the least important variable.
- The PCA algorithm is based on some mathematical concepts such as:
 - Variance and Covariance
 - Eigen values and Eigen vectors



Principal Component Analysis Algorithm

- **Some common terms used in PCA algorithm:**
- **Dimensionality:** It is the number of features or variables present in the given dataset. More easily, it is the number of columns present in the dataset.
- **Correlation:** It signifies that how strongly two variables are related to each other. Such as if one changes, the other variable also gets changed. The correlation value ranges from -1 to +1. Here, -1 occurs if variables are inversely proportional to each other, and +1 indicates that variables are directly proportional to each other.
- **Orthogonal:** It defines that variables are not correlated to each other, and hence the correlation between the pair of variables is zero.
- **Eigen vectors:** If there is a square matrix M , and a non-zero vector v is given. Then v will be eigenvector if Av is the scalar multiple of v .
- **Covariance Matrix:** A matrix containing the covariance between the pair of variables is called the Covariance Matrix.
- **Principal Components in PCA**
- As described above, the transformed new features or the output of PCA are the Principal Components. The number of these PCs are either equal to or less than the original features present in the dataset. Some properties of these principal components are given below:
 - The principal component must be the linear combination of the original features.
 - These components are orthogonal, i.e., the correlation between a pair of variables is zero.
 - The importance of each component decreases when going to 1 to n , it means the 1 PC has the most importance, and n PC will have the least importance.

Principal Component Analysis Algorithm

■ Steps for PCA algorithm

1. Calculate Mean

In this step, we will standardize our dataset. Such as in a particular column, the features with high variance are more important compared to the features with lower variance. If the importance of features is independent of the variance of the feature, then we will divide each data item in a column with the standard deviation of the column. Here we will name the matrix as Z .

2. Calculating the Covariance of Z

To calculate the covariance of Z , we will take the matrix Z , and will transpose it. After transpose, we will multiply it by Z . The output matrix will be the Covariance matrix of Z .

3. Calculating the Eigen Values and Eigen Vectors

Now we need to calculate the eigenvalues and eigenvectors for the resultant covariance matrix Z . Eigenvectors or the covariance matrix are the directions of the axes with high information. And the coefficients of these eigenvectors are defined as the eigenvalues.

4. Sorting the Eigen Vectors

In this step, we will take all the eigenvalues and will sort them in decreasing order, which means from largest to smallest. And simultaneously sort the eigenvectors accordingly in matrix P of eigenvalues. The resultant matrix will be named as P^* .

5. Computation of first Principal Component

6. Geometrical meaning of first principal components

Principal Component Analysis Algorithm

Suppose we have marks of 5 students in two subjects.

Student	Math (X_1)	Physics (X_2)
A	2	1
B	4	3
C	6	5
D	8	7
E	10	9

We will reduce 2 dimensions into 1 principal component.

Step 1: Construct Data Matrix

$$X = \begin{bmatrix} 2 & 1 \\ 4 & 3 \\ 6 & 5 \\ 8 & 7 \\ 10 & 9 \end{bmatrix}$$

Step 2: Calculate Mean

Mean of Math:

$$\bar{X}_1 = \frac{2 + 4 + 6 + 8 + 10}{5} = 6$$

Mean of Physics:

$$\bar{X}_2 = \frac{1 + 3 + 5 + 7 + 9}{5} = 5$$

Mean vector:

$$\mu = (6, 5)$$

Principal Component Analysis Algorithm

Step 3: Center the Data

$$X_{centered} = X - \mu$$

Subtract mean from each value.

Student	$(X_1 - 6)$	$(X_2 - 5)$
A	-4	-4
B	-2	-2
C	0	0
D	2	2
E	4	4

Centered matrix:

$$A = \begin{bmatrix} -4 & -4 \\ -2 & -2 \\ 0 & 0 \\ 2 & 2 \\ 4 & 4 \end{bmatrix}$$

Step 4: Find Covariance Matrix

$$Cov(A) = \frac{1}{n-1} A^T A$$

Here $n=5$

First transpose:

$$A^T = \begin{bmatrix} -4 & -2 & 0 & 2 & 4 \\ -4 & -2 & 0 & 2 & 4 \end{bmatrix}$$

$$A^T A = \begin{bmatrix} 40 & 40 \\ 40 & 40 \end{bmatrix}$$

Divide by $n-1=4$:

$$Cov = \begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix}$$

Principal Component Analysis Algorithm

Step 5: Compute Eigenvalues

$$|C - \lambda I| = 0$$

$$(10 - \lambda)^2 - 100 = 0$$

$$100 - 20\lambda + \lambda^2 - 100 = 0$$

$$\begin{vmatrix} 10 - \lambda & 10 \\ 10 & 10 - \lambda \end{vmatrix} = 0$$

$$\lambda^2 - 20\lambda = 0$$

$$\lambda(\lambda - 20) = 0$$

Eigenvalues: $\lambda_1=20$, $\lambda_2=0$

Step 6: Find Eigenvectors

For largest eigenvalue: $\lambda_1=20$

$$(C - \lambda I)v = 0$$

$$C - 20I = \begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix} - \begin{bmatrix} 20 & 0 \\ 0 & 20 \end{bmatrix} = \begin{bmatrix} -10 & 10 \\ 10 & -10 \end{bmatrix}$$

Assume eigenvector $v = \begin{bmatrix} x \\ y \end{bmatrix}$

$$\begin{bmatrix} -10 & 10 \\ 10 & -10 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

First Row: $(-10)(x) + (10)(y) = 0$ $x=y$ or $y=x$ **Second Row:** $10x - 10y = 0$ $x=y$

Both equations give the same result.

Normalize it:

$$\|v\| = \sqrt{1^2 + 1^2} = \sqrt{2} \quad v = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad v_1 = \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix}$$

This is the First Principal Component.

- C = covariance matrix
- λ = eigenvalue
- I = identity matrix
- v = eigenvector

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Principal Component Analysis Algorithm

Step 7: Project Data onto Principal Component

Formula

$$Z = Av_1$$

For Student A

$$\begin{aligned} &(-4)(0.707) + (-4)(0.707) \\ &= -5.656 \end{aligned}$$

Student B

$$\begin{aligned} &(-2)(0.707) + (-2)(0.707) \\ &= -2.828 \end{aligned}$$

Student C

0

Student D

$$\begin{aligned} &(2)(0.707) + (2)(0.707) \\ &= 2.828 \end{aligned}$$

Student E

$$\begin{aligned} &(4)(0.707) + (4)(0.707) \\ &= 5.656 \end{aligned}$$

Principal Component Analysis Algorithm

Final Reduced Data

Student	PC1
A	-5.656
B	-2.828
C	0
D	2.828
E	5.656

Now the original 2D data became 1D data.

Variance Explained

Total variance: $20+0=20$

Variance explained by PC

$$\frac{20}{20} \times 100 = 100\%$$

So the first principal component keeps all information.

Geometrical Meaning

PCA finds the direction where data spreads most.

In this dataset:

- Math and Physics increase together
- So data lies approximately on a straight line

PCA rotates axes and chooses the best axis.

Unsupervised Learning: Association

Association rule learning is an unsupervised learning technique, which finds interesting relationships, correlations or patterns within large sets of data items. The main aim of this learning algorithm is to find the dependency of one data item on another data item and map those variables accordingly so that it can generate maximum profit. This algorithm is mainly applied in **Market Basket analysis, Web usage mining, continuous production**, etc.

Some popular algorithms of Association rule learning are **Apriori Algorithm, Eclat, FP-growth algorithm**.



Customer 1



Customer 2



Customer 3



Customer n

Unsupervised Learning: Association

- Association rules are a fundamental concept used to find relationships, correlations or patterns within large sets of data items. They describe how often itemsets occur together in transactions and express implications of the form.

$$X \rightarrow Y$$

Where X and Y are disjoint sets of items. This rule suggests that when items in X appear, items in Y tend to appear as well. Association rules originated from market basket analysis and help retailers and analysts understand customer behavior by discovering item associations in transaction data. For example, a rule stating

$$\{\text{Bread, Butter}\} \rightarrow \{\text{Milk}\}$$

Indicates that customers who buy bread and butter also tend to buy milk.

Key Components

- Antecedent (X):** The "if" part representing one or more items found in transactions.
- Consequent (Y):** The "then" part, representing the items likely to be purchased when antecedent items appear.

Rules are evaluated based on metrics that quantify their strength and usefulness:

Rule Evaluation Metrics

- Support:** Fraction of transactions containing the itemsets in both X and Y .

$$\text{Support}(X \rightarrow Y) = \frac{\text{Number of transactions with } (X \cup Y)}{\text{Total number of transactions}}$$

Support measures how frequently the combination appears in the data.

Unsupervised Learning: Association

- Association rules are a fundamental concept used to find relationships, correlations or patterns within large sets of data items. They describe how often itemsets occur together in transactions and express implications of the form.

$$X \rightarrow Y$$

Where X and Y are disjoint sets of items. This rule suggests that when items in X appear, items in Y tend to appear as well. Association rules originated from market basket analysis and help retailers and analysts understand customer behavior by discovering item associations in transaction data. For example, a rule stating

$$\{\text{Bread, Butter}\} \rightarrow \{\text{Milk}\}$$

Indicates that customers who buy bread and butter also tend to buy milk.

Key Components

- Antecedent (X):** The "if" part representing one or more items found in transactions.
- Consequent (Y):** The "then" part, representing the items likely to be purchased when antecedent items appear.

Rules are evaluated based on metrics that quantify their strength and usefulness:

Rule Evaluation Metrics

- Support:** Fraction of transactions containing the itemsets in both X and Y .

$$\text{Support}(X \rightarrow Y) = \frac{\text{Number of transactions with } (X \cup Y)}{\text{Total number of transactions}}$$

Support measures how frequently the combination appears in the data.

Unsupervised Learning: Association

2. Confidence: Probability that transactions with X also include Y.

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

Confidence measures the reliability of the inference.

3. Lift: The ratio of observed support to that expected if X and Y were independent.

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$$

- Lift > 1 implies a positive association — items occur together more than expected.
- Lift = 1 implies independence.
- Lift < 1 implies a negative association.

Considering the rule {Milk, Diaper} → {Beer}

Calculations:

- Support = $\frac{2}{5} = 0.4$
- Confidence = $\frac{2}{3} \approx 0.67$
- Lift = $\frac{0.67}{0.4} = 1.11$ (positive association)

Transaction ID	Items
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Apriori Algorithm

The Apriori algorithm is a classic algorithm used in data mining for learning association rules. It is designed to operate on databases containing transactions, such as those found in retail environments. The primary goal of the Apriori algorithm is to identify frequent itemsets (sets of items that appear together frequently) and then derive association rules from these itemsets.

Key Concepts:

Itemset: A collection of one or more items. For example, in a supermarket, an itemset could be {Milk, Bread}.

Support: The proportion of transactions in the database in which the itemset appears. For example, if {Milk, Bread} appears in 30 out of 100 transactions, its support is 30%.

Frequent Itemset: An itemset whose support is greater than or equal to a specified minimum support threshold.

Association Rule: An implication of the form $X \rightarrow Y$, where X and Y are itemsets. For example, $\{\text{Milk}\} \rightarrow \{\text{Bread}\}$ means that if Milk is bought, Bread is also likely to be bought.

Confidence: The proportion of transactions containing X that also contain Y . It is calculated as the support of $X \cup Y$ divided by the support of X .

Applications:

- Market basket analysis.
- Recommendation systems.
- Cross-selling strategies.
- Inventory management.

The Apriori algorithm is foundational in the field of association rule mining and has inspired many subsequent algorithms and improvements.

Apriori Algorithm

- Apriori algorithm refers to the algorithm which is used to calculate the association rules between objects. It means how two or more objects are related to one another. In other words, we can say that the apriori algorithm is an association rule learning that analyzes that people who bought product A also bought product B.
- **What is Apriori Algorithm?**
- Apriori algorithm refers to an algorithm that is used in mining frequent products sets and relevant association rules. Generally, the apriori algorithm operates on a database containing a huge number of transactions. For example, the items customers buy at a Big Bazar. Apriori algorithm helps the customers to buy their products with ease and increases the sales performance of the particular store.
- **Components of Apriori algorithm**

The given three components comprise the apriori algorithm.

1.Support

2.Confidence

3.Lift

Let's take an example to understand this concept.

you need a huge database containing a large no of transactions. Suppose you have 4000 customers transactions in a Big Bazar. You have to calculate the Support, Confidence, and Lift for two products, and you may say Biscuits and Chocolate. This is because customers frequently buy these two items together.

Apriori Algorithm-Example

- Out of 4000 transactions, 400 contain Biscuits, whereas 600 contain Chocolate, and these 600 transactions include a 200 that includes Biscuits and chocolates. Using this data, we will find out the support, confidence, and lift.

Support

- Support refers to the default popularity of any product. You find the support as a quotient of the division of the number of transactions comprising that product by the total number of transactions. Hence, we get
- Support (Biscuits) = (Transactions relating biscuits) / (Total transactions)
- = $400/4000 = 10$ percent.

Confidence

- Confidence refers to the possibility that the customers bought both biscuits and chocolates together. So, you need to divide the number of transactions that comprise both biscuits and chocolates by the total number of transactions to get the confidence.
- Hence, Confidence = (Transactions relating both biscuits and Chocolate) / (Total transactions involving Biscuits)
= $200/400$
= 50 percent.

It means that 50 percent of customers who bought biscuits bought chocolates also.

Lift

Consider the above example; lift refers to the increase in the ratio of the sale of chocolates when you sell biscuits. The mathematical equations of lift are given below.

$$\text{Lift} = (\text{Confidence (Biscuits - chocolates)}) / (\text{Support (Biscuits)})$$
$$= 50/10 = 5$$

It means that the probability of people buying both biscuits and chocolates together is five times more than that of purchasing the biscuits alone. If the lift value is below one, it requires that the people are unlikely to buy both the items together. Larger the value, the better is the combination.

How does the Apriori Algorithm work?

We will understand this algorithm with the help of an example

Consider a Big Bazar scenario where the product set is $P = \{\text{Rice, Pulse, Oil, Milk, Apple}\}$. The database comprises six transactions where 1 represents the presence of the product and 0 represents the absence of the product.

Transac tion ID	Rice	Pulse	Oil	Milk	Apple
t1	1	1	1	0	0
t2	0	1	1	1	0
t3	0	0	0	1	1
t4	1	1	0	1	0
t5	1	1	1	0	1
t6	1	1	1	1	1

Product	Frequency (Number of transactions)
Rice (R)	4
Pulse(P)	5
Oil(O)	4
Milk(M)	4

The Apriori Algorithm makes the given assumptions:

Step:1

Make a frequency table of all the products that appear in all the transactions. Now, short the frequency table to add only those products with a threshold support level of over 50 percent. We find the given frequency table. The above table (table -2) indicated the products frequently bought by the customers.

How does the Apriori Algorithm work?

Step 2

Create pairs of products such as RP, RO, RM, PO, PM, OM. You will get the given frequency table.

Itemset	Frequency (Number of transactions)
RP	4
RO	3
RM	2
PO	4
PM	3
OM	2

Itemset	Frequency (Number of transactions)
RPO	4
POM	3

If you implement the threshold assumption, you can figure out that the customers' set of three products is RPO.

Step 3

Implementing the same threshold support of 50 percent and consider the products that are more than 50 percent. In our case, it is more than 3

Thus, we get RP, RO, PO, and PM

Step 4

Now, look for a set of three products that the customers buy together. We get the given combination.

RP and RO give RPO

PO and PM give POM

Step 5

Calculate the frequency of the two itemsets, and you will get the given frequency table(table 2).

How does the Apriori Algorithm work?

Steps of the Apriori Algorithm:

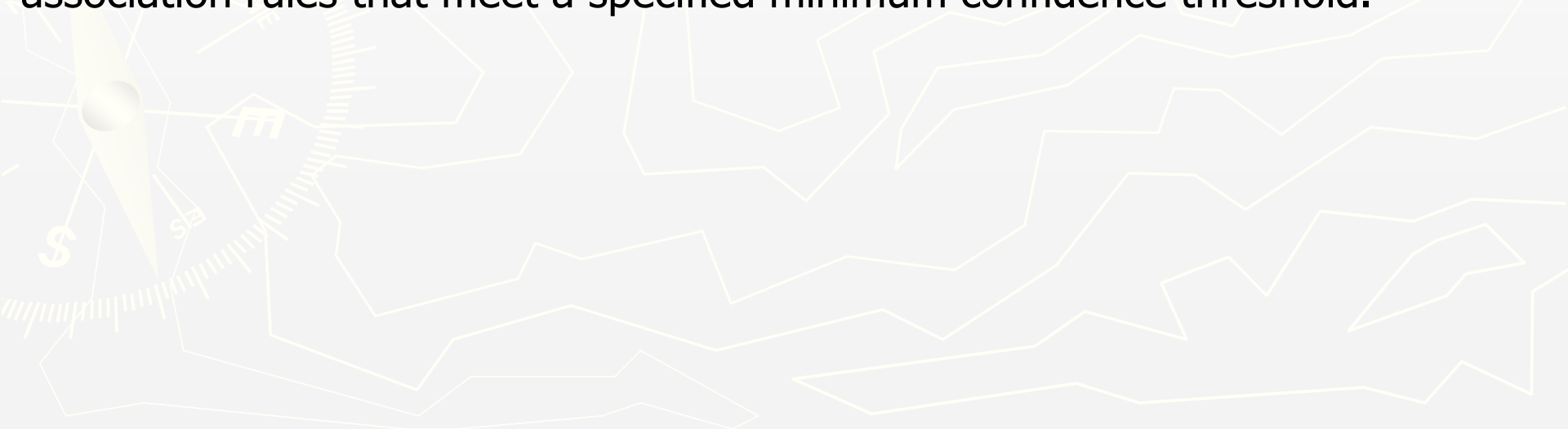
Generate Candidate Itemsets: Start by generating all possible 1-itemsets (itemsets with a single item) and calculate their support. Prune those that do not meet the minimum support threshold.

Join Step: Generate candidate k -itemsets by joining $(k-1)$ -itemsets that share the first $(k-2)$ items.

Prune Step: Prune the candidate k -itemsets by removing those that contain any $(k-1)$ -subset that is not frequent.

Repeat: Repeat the join and prune steps until no more frequent itemsets can be found.

Generate Association Rules: From the frequent itemsets, generate association rules that meet a specified minimum confidence threshold.



Apriori Algorithm Solved Example 3 Association Rule Mining

TID	List of Items IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

- Consider the following transactions.
- Apply the association rule mining to get the association rules with **min support of 2** and **confidence of 50%**

Apriori Algorithm Solved Example 3 Association Rule Mining

TID	List of Items IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

1-item Sets	Frequency
I1	6
I2	7
I3	6
I4	2
I5	2

Frequent 1-item Sets	Frequency
I1	6
I2	7
I3	6
I4	2
I5	2

Apriori Algorithm Solved Example 3 Association Rule Mining

TID	List of Items IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

2-item Sets	Frequency
I1, I2	4
I1, I3	4
I1, I4	1
I1, I5	2
I2, I3	4
I2, I4	2
I2, I5	2
I3, I4	0
I3, I5	1
I4, I5	0

Frequent 2-item Sets	Frequency
I1, I2	4
I1, I3	4
I1, I5	2
I2, I3	4
I2, I4	2
I2, I5	2

Apriori Algorithm Solved Example 3 Association Rule Mining

TID	List of Items IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

3-item Sets	Frequency
I1, I2, I3	2
I1, I2, I4	1
I1, I2, I5	2
I1, I3, I4	0
I1, I3, I5	1
I1, I4, I5	0
I2, I3, I4	0
I2, I3, I5	1
I2, I4, I5	0
I3, I4, I5	0

I1, I2, I3, I4, I5

Frequent 3-item Sets	Frequency
I1, I2, I3	2
I1, I2, I5	2

Apriori Algorithm Solved Example 3 Association Rule Mining

Frequent 3-Item Set = $I \Rightarrow \{1, 2, 3\}$ and $\{1, 2, 5\}$

- $\text{Min_Support} = 2 = 2/9 = 22.22\%$ and $\text{Min_Confidence} = 50\%$

Non-Empty subset are

$\{\{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}\}$

$\{\{1\}, \{2\}, \{5\}, \{1, 2\}, \{1, 5\}, \{2, 5\}\}$

How to form Association Rule...?

For every non-empty subset S of I , the association rule is,

$$S \rightarrow (I-S)$$

If $\text{support}(I) / \text{support}(S) \geq \text{min_confidence}$

Apriori Algorithm

Frequent 3-Item Set = $I \Rightarrow \{1, 2, 3\}$

Non-Empty subset are

$\{\{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}\}$

Rule 1: $\{1\} \rightarrow \{2, 3\}$ $\{S= 22.22 \%, C=33.34\%\}$

Support = $2/9 = 22.22\%$

Confidence = $\text{Support}(1, 2, 3) / \text{Support}(1) = \frac{2/9}{6/9} = 2/6 = 33.34\% < 50\%$ Invalid Rule

Rule 2: $\{2\} \rightarrow \{1, 3\}$ $\{S= 22.22 \%, C=28.57\%\}$

Support = $2/9 = 22.22 \%$

Confidence = $\text{Support}(1, 2, 3) / \text{Support}(2) = \frac{2/9}{7/9} = 2/7 = 28.57\% < 50\%$ Invalid Rule

Items	Frequency
I1	6
I2	7
I3	6
I1, I2	4
I1, I3	4
I2, I3	4

Apriori Algorithm

Frequent 3-Item Set = $I \Rightarrow \{1, 2, 3\}$

Non-Empty subset are

$\{\{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}\}$

Rule 3: $\{3\} \rightarrow \{1, 2\}$ {S= 22.22 %, C=33.33%}

Support = $2/9 = 22.22\%$

Confidence = $\text{Support}(1, 2, 3) / \text{Support}(3) = \frac{2/9}{6/9} = 2/6 = 33.33\% < 50\%$

Rule 4: $\{1, 2\} \rightarrow \{3\}$ {S= 22.22 %, C=50%}

Support = $2/9 = 22.22\%$

Confidence = $\text{Support}(1, 2, 3) / \text{Support}(1, 2) = \frac{2/9}{4/9} = 2/4 = 50\% \geq 50\%$

Items	Frequency
I1	6
I2	7
I3	6
I1, I2	4
I1, I3	4
I2, I3	4

Invalid Rule

Valid Rule

Apriori Algorithm

Frequent 3-Item Set = $I \Rightarrow \{1, 2, 3\}$

Non-Empty subset are

$\{\{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}\}$

Rule 5: $\{1, 3\} \rightarrow \{2\}$ {S= 22.22 %, C=50%}

Support = $2/9 = 22.22\%$

Confidence = $\text{Support}(1, 2, 3) / \text{Support}(1, 3) = \frac{2/9}{4/9} = 2/4 = 50\% \geq 50\%$

Items	Frequency
I1	6
I2	7
I3	6
I1, I2	4
I1, I3	4
I2, I3	4

Valid Rule

Rule 6: $\{2, 3\} \rightarrow \{1\}$ {S= 22.22 %, C=66.67%}

Support = $2/9 = 22.22\%$

Confidence = $\text{Support}(1, 2, 3) / \text{Support}(2, 3) = \frac{2/9}{4/9} = 2/4 = 50\% \geq 50\%$

Valid Rule

Apriori Algorithm

Frequent 3-Item Set = I $\Rightarrow$ {1, 2, 5}

Non-Empty subset are

{1}, {2}, {5}, {1, 2}, {1, 5}, {2, 5}

Rule 1: {1} $\rightarrow$ {2, 5} {S= 22.22 %, C=33.34%}

Support = $2/9 = 22.22 \%$

Confidence = $\text{Support}(1, 2, 5) / \text{Support}(1) = \frac{2/9}{6/9} = 2/6 = 33.34\% < 50\%$

Items	Frequency
I1	6
I2	7
I5	2
I1, I2	4
I1, I5	2
I2, I5	2

Invalid Rule

Rule 2: {2} $\rightarrow$ {1, 5} {S= 22.22 %, C=28.57%}

Support = $2/9 = 22.22 \%$

Confidence = $\text{Support}(1, 2, 5) / \text{Support}(2) = \frac{2/9}{7/9} = 2/7 = 28.57\% < 50\%$

Invalid Rule

Apriori Algorithm

Frequent 3-Item Set = $I \Rightarrow \{1, 2, 5\}$

Non-Empty subset are

$\{\{1\}, \{2\}, \{5\}, \{1, 2\}, \{1, 5\}, \{2, 5\}\}$

Rule 3: $\{5\} \rightarrow \{1, 2\}$ $\{S=22.22\%, C=100\%\}$

Support = $2/9 = 22.22\%$

Confidence = $\text{Support}(1, 2, 5) / \text{Support}(5) = \frac{2/9}{2/9} = 2/2 = 100\% \geq 50\%$

Valid Rule

Rule 4: $\{1, 2\} \rightarrow \{5\}$ $\{S=22.22\%, C=50\%\}$

Support = $2/9 = 22.22\%$

Confidence = $\text{Support}(1, 2, 5) / \text{Support}(1, 2) = \frac{2/9}{4/9} = 2/4 = 50\% \geq 50\%$

Valid Rule

Items	Frequency
I1	6
I2	7
I5	2
I1, I2	4
I1, I5	2
I2, I5	2

Apriori Algorithm

Frequent 3-Item Set = $I \Rightarrow \{1, 2, 5\}$

Non-Empty subset are

$\{\{1\}, \{2\}, \{5\}, \{1, 2\}, \{1, 5\}, \{2, 5\}\}$

Rule 5: $\{1, 5\} \rightarrow \{2\}$ {S=22.22%, C=100%}

Support = $2/9 = 22.22\%$

Confidence = $\text{Support}(1, 2, 5) / \text{Support}(1, 5) = \frac{2/9}{2/9} = 2/2 = 100\% \geq 50\%$

Rule 6: $\{2, 5\} \rightarrow \{1\}$ {S=22.22%, C=100%}

Support = $2/9 = 22.22\%$

Confidence = $\text{Support}(1, 2, 5) / \text{Support}(2, 5) = \frac{2/9}{2/9} = 2/2 = 100\% \geq 50\%$

Items	Frequency
I1	6
I2	7
I5	2
I1, I2	4
I1, I5	2
I2, I5	2

Valid Rule

Valid Rule

Apriori Algorithm

Problem Setup

We are given the following transaction database:

Transaction ID	Items Purchased
T1	{A, B, C}
T2	{A, C}
T3	{A, D}
T4	{B, E, F}
T5	{A, C, D}

Thresholds:

Minimum Support Threshold: 40% (i.e., an itemset must appear in at least 2 out of 5 transactions to be considered frequent).

Minimum Confidence Threshold: 70% (for association rules).

Apriori Algorithm

Step 1: Generate 1-Itemsets and Calculate Support

Itemset	Support Count	Support (%)
{A}	4	$4/5 = 80\%$
{B}	2	$2/5 = 40\%$
{C}	3	$3/5 = 60\%$
{D}	2	$2/5 = 40\%$
{E}	1	$1/5 = 20\%$
{F}	1	$1/5 = 20\%$

Pruning: Remove itemsets with support < 40%. So, {E} and {F} are pruned.

Frequent 1-Itemsets: {A}, {B}, {C}, {D}

Apriori Algorithm

Step 2: Generate 2-Itemsets and Calculate Support

Now, we generate all possible 2-itemsets by joining frequent 1-itemsets and calculate their support:

Keep count ≥ 2

Itemset	Support Count	Support (%)
{A, B}	1	$1/5 = 20\%$
{A, C}	3	$3/5 = 60\%$
{A, D}	2	$2/5 = 40\%$
{B, C}	1	$1/5 = 20\%$
{B, D}	0	$0/5 = 0\%$
{C, D}	1	$1/5 = 20\%$

Pruning: Remove itemsets with support $< 40\%$. So, {A, B}, {B, C}, {B, D}, and {C, D} are pruned.

Frequent 2-Itemsets: {A, C}, {A, D}

Apriori Algorithm

Step 3: Generate 3-Itemsets and Calculate Support

Next, we generate 3-itemsets by joining frequent 2-itemsets and calculate their support:

Itemset	Support Count	Support (%)
{A, C, D}	1	$1/5 = 20\%$

Pruning: Remove itemsets with support < 40%. So, {A, C, D} is pruned.

Frequent 3-Itemsets: None

Step 4: Generate Association Rules

From the frequent itemsets, we generate association rules and calculate their confidence.

Frequent Itemsets:

- {A}, {B}, {C}, {D}
- {A, C}, {A, D}

Rule Generation:

1. From {A, C}:

○ Rule: {A} → {C}

■ Confidence = $\text{Support}(\{A, C\}) / \text{Support}(\{A\}) = (3/5) / (4/5) = 75\%$

○ Rule: {C} → {A}

■ Confidence = $\text{Support}(\{A, C\}) / \text{Support}(\{C\}) = (3/5) / (3/5) = 100\%$

•Support(AC) = 3

•Support(A) = 4

Confidence=3/4=75% ✓✓ Strong rule



🔥 Very strong

Apriori Algorithm

2. From {A, D}:

- Rule: {A} → {D}
 - Confidence = $\text{Support}(\{A, D\}) / \text{Support}(\{A\}) = (2/5) / (4/5) = 50\%$ ✕
- Rule: {D} → {A}
 - Confidence = $\text{Support}(\{A, D\}) / \text{Support}(\{D\}) = (2/5) / (2/5) = 100\%$

🔥 Strong rule

Pruning Rules:

- Keep rules with confidence $\geq 70\%$.
- Pruned rules: {A} → {D} (50% confidence).

Final Association Rules:

1. {A} → {C} (Confidence = 75%)
2. {C} → {A} (Confidence = 100%)
3. {D} → {A} (Confidence = 100%)

Apriori Algorithm

Final Results

Frequent Itemsets:

- 1-Itemsets: {A}, {B}, {C}, {D}
- 2-Itemsets: {A, C}, {A, D}

Association Rules:

1. {A} $\rightarrow$ {C} (75% confidence)
2. {C} $\rightarrow$ {A} (100% confidence)
3. {D} $\rightarrow$ {A} (100% confidence)

Key Insights

1. Why {A,C,D} is pruned?

Because:

- CD is **not frequent**
- So by Apriori:

If subset is not frequent $\rightarrow$ superset cannot be frequent

2. Important Observation

Even though:

- A appears many times
- D appears few times

👉 **D $\rightarrow$ A is very strong (100%)**

Final Strong Rules ($\geq 70\%$)

- ✓ A $\rightarrow$ C (75%)
- ✓ C $\rightarrow$ A (100%)
- ✓ D $\rightarrow$ A (100%)

Final Answer

✓ Frequent Itemsets:

- L1 = {A, B, C, D}
- L2 = {A,C}, {A,D}

✓ Strong Rules:

- A $\rightarrow$ C
- C $\rightarrow$ A
- D $\rightarrow$ A

ECLAT Algorithm

The ECLAT algorithm stands for **Equivalence Class Clustering and bottom-up Lattice Traversal**

- It is one of the popular methods of Association Rule mining
- It is an improved alternative to the Apriori algorithm by providing better scalability and computational efficiency. While Apriori uses a horizontal format where each transaction is a row and it follows a breadth-first search (BFS) strategy, this means it scans the database multiple times to find frequent item combinations, ECLAT on the other hand uses a vertical format where each item is linked to a list of transaction IDs (TIDs). It uses a depth-first search (DFS) strategy which requires fewer scans and makes it faster and more memory-efficient. This vertical approach significantly reduces the number of database scans making ECLAT faster and more memory-efficient especially for large datasets.
- **How the algorithm work? :**

The basic idea is to use Transaction Id Sets(tidsets) intersections to compute the support value of a candidate and avoiding the generation of subsets which do not exist in the prefix tree. In the first call of the function, all single items are used along with their tidsets. Then the function is called recursively and in each recursive call, each item-tidset pair is verified and combined with other item-tidset pairs. This process is continued until no candidate item-tidset pairs can be combined.

- Let us now understand the above stated working with an example:- Consider the following transactions record:-

ECLAT Algorithm

Transaction Id	Bread	Butter	Milk	Coke	Jam
T1	1	1	0	0	1
T2	0	1	0	1	0
T3	0	1	1	0	0
T4	1	1	0	1	0
T5	1	0	1	0	0
T6	0	1	1	0	0
T7	1	0	1	0	0
T8	1	1	1	0	1
T9	1	1	1	0	0

Step 1: Create the Tidset

The above-given data is a boolean matrix where for each cell (i, j), the value denotes whether the j'th item is included in the ith transaction or not. 1 means true while 0 means false. We now call the function for the first time and arrange each item with it's tidset in a tabular fashion: **k = 1, minimum support = 2**

Item	Tidset
Bread	{T1, T4, T5, T7, T8, T9}
Butter	{T1, T2, T3, T4, T6, T8, T9}
Milk	{T3, T5, T6, T7, T8, T9}
Coke	{T2, T4}
Jam	{T1, T8}

ECLAT Algorithm

Step 2: Calculate the Support of Itemsets by Intersecting Tidsets

ECLAT then proceeds by recursively combining the tidsets. The support of an itemset is determined by the intersection of tidsets. We now recursively call the function till no more item-tidset pairs can be combined: **k = 2**

Item	Tidset
{Bread, Butter}	{T1, T4, T8, T9}
{Bread, Milk}	{T5, T7, T8, T9}
{Bread, Coke}	{T4}
{Bread, Jam}	{T1, T8}
{Butter, Milk}	{T3, T6, T8, T9}
{Butter, Coke}	{T2, T4}
{Butter, Jam}	{T1, T8}
k = 3 {Milk, Jam}	{T8}

Step 3: Recursive Call and Generation of Larger Itemsets

The algorithm continues recursively by combining pairs of itemsets (k-itemsets) checking the support by intersecting the tidsets. The recursion continues until no further frequent itemsets can be generated. Now **k = 3**

Item	Tidset
{Bread, Butter, Milk}	{T8, T9}
{Bread, Butter, Jam}	{T1, T8}

ECLAT Algorithm

Step 4: Stop When No More Frequent Itemsets Can Be Found

The algorithm stops once no more itemset combinations meet the minimum support threshold. $k = 4$

Item	Tidset
{Bread, Butter, Milk, Jam}	{T8}

We stop at $k = 4$ because there are no more item-tidset pairs to combine. Since minimum support = 2, we conclude the following rules from the given dataset:-

Items Bought	Recommended Products
Bread	Butter
Bread	Milk
Bread	Jam
Butter	Milk
Butter	Coke
Butter	Jam
Bread and Butter	Milk
Bread and Butter	Jam

ECLAT Algorithm

Problem

Consider the following transaction database

TID	Items Purchased
T1	A, B, D
T2	B, C, E
T3	A, B, C, E
T4	B, E
T5	A, B, C, E

Minimum support count: minsup=2

Use the **ECLAT Algorithm** to find all frequent itemsets.

Step 1: Convert into Vertical Data Format

ECLAT converts transactions into **TID sets**.

Item	TID Set	Support
A	{T1, T3, T5}	3
B	{T1, T2, T3, T4, T5}	5
C	{T2, T3, T5}	3
D	{T1}	1
E	{T2, T3, T4, T5}	4

ECLAT Algorithm

Since minimum support = 2, Item D is removed.

Frequent 1-Itemsets

Itemset	Support
A	3
B	5
C	3
E	4

Step 2: Generate Frequent 2-Itemsets

We intersect TID sets.

For (A,B)

$A = \{T1, T3, T5\}$ $B = \{T1, T2, T3, T4, T5\}$ $A \cap B = \{T1, T3, T5\}$ Support: 3 Frequent

For (A,C) $A \cap C = \{T3, T5\}$ Support: 2 Frequent

For (A,E) $A \cap E = \{T3, T5\}$ Support: 2 Frequent

For (B,C) $B \cap C = \{T2, T3, T5\}$ Support: 3 Frequent

For (B,E) $B \cap E = \{T2, T3, T4, T5\}$ Support: 4 Frequent

For (C,E) $C \cap E = \{T2, T3, T5\}$ Support: 3 Frequent

ECLAT Algorithm

Frequent 2-Itemsets

Itemset	Support
(A,B)	3
(A,C)	2
(A,E)	2
(B,C)	3
(B,E)	4
(C,E)	3

Step 3: Generate Frequent 3-Itemsets

For (A,B,C): $(A,B) = \{T1, T3, T5\}, C = \{T2, T3, T5\}, (A \cap B) \cap C = \{T3, T5\}$ **Sup: 2 Freq**

For (A,B,E): $E = \{T2, T3, T4, T5\}, (A \cap B) \cap E = \{T3, T5\}$ **Support: 2 Frequent**

For (B,C,E): $(B,C) = \{T2, T3, T5\}, E = \{T2, T3, T4, T5\}, (B \cap C) \cap E = \{T2, T3, T5\}$ **Sup: 3 Freq**

Frequent 3-Itemsets

Itemset	Support
(A,B,C)	2
(A,B,E)	2
(A,C,E)	2
(B,C,E)	3

ECLAT Algorithm

Step 4: Generate Frequent 4-Itemsets

For (A,B,C,E): $(A,B,C)=\{T3,T5\}$, $E=\{T2,T3,T4,T5\}$, $((A\cap B)\cap C)\cap E=\{T3,T5\}$

Support: 2 Frequent

Final Frequent Itemsets

Itemset	Support
A	3
B	5
C	3
E	4
AB	3
AC	2
AE	2
BC	3
BE	4
CE	3
ABC	2
ABE	2
ACE	2
BCE	3
ABCE	2

ECLAT Algorithm

Advantages over Apriori algorithm:-

- **Memory Requirements:** Since the ECLAT algorithm uses a Depth-First Search approach, it uses less memory than Apriori algorithm.
- **Speed:** The ECLAT algorithm is typically faster than the Apriori algorithm.
- **Number of Computations:** The ECLAT algorithm does not involve the repeated scanning of the data to compute the individual support values.
- **Efficient Support Counting:** Support is calculated using simple set intersections. This is computationally efficient.
- **Good for Dense Datasets**

Works very well when:

- ✓ transactions contain many items
- ✓ many frequent itemsets exist
- **Less Candidate Generation:**
Compared to Apriori: fewer candidate itemsets are generated
pruning is more effective
- **Better Scalability:** Can handle large transactional databases more efficiently than some traditional algorithm

Disadvantages of ECLAT

1. High Memory Usage
2. Large TID Sets
3. Not Good for Sparse Datasets
4. Complex Implementation

Frequent Pattern Growth Algorithm

The FP-Growth (Frequent Pattern Growth) algorithm efficiently mines frequent itemsets from large transactional datasets. Unlike the Apriori algorithm which suffers from high computational cost due to candidate generation and multiple database scans. FP-Growth avoids these inefficiencies by compressing the data into an FP-Tree (Frequent Pattern Tree) and extracts patterns directly from it. Here's how it works in simple terms:

How FP-Growth Works

1.Data Compression: First FP-Growth compresses the dataset into a smaller structure called the Frequent Pattern Tree (FP-Tree). This tree stores information about item sets (collections of items) and their frequencies without need to generate candidate sets like Apriori does.

2.Mining the Tree: The algorithm then examines this tree to identify patterns that appear frequently based on a minimum support threshold. It does this by breaking the tree down into smaller "conditional" trees for each item making the process more efficient.

3.Generating Patterns: Once the tree is built and analyzed the algorithm generates the frequent patterns (itemsets) and the rules that describe relationships between items.

Frequent Pattern Growth Algorithm

Imagine you're organizing a party and want to know popular food combinations without asking every guest repeatedly.

1. List food items each guest brought transactions.
2. Count items and remove infrequent ones filter by support.
3. Group items in order of popularity and create a tree where paths represent common combinations.
4. Instead of repeatedly asking guests you explore this tree to discover patterns. For example, you might find that pizza and pasta often come together or that cake and pasta are also a common pair

Problem Statement: Consider a small grocery store transaction dataset. Each entry shows the set of items purchased together by a customer:

Transaction ID	Sorted Items by Frequency
T1	Bread, Milk, Butter
T2	Bread , Butter
T3	Bread, Milk
T4	Milk , Butter
T5	Bread

We apply the FP-Growth algorithm to identify frequent itemsets (groups of items frequently bought together), using a minimum support count of 2.

Frequent Pattern Growth Algorithm

Step 1: Compute Item Frequencies

Scan the entire dataset one time to determine how often each item appears.

Item	Support Count
Milk	3
Bread	4
Butter	3

All items meet the minimum support threshold (≥ 2), so none are removed.

Step 2: Order Items in Each Transaction by Frequency

Next, sort every transaction in descending order based on how frequently the items appear overall.

Sorting by frequency helps group common items together, allowing the FP-tree to be compressed efficiently.

Transaction ID	Sorted Items by Frequency
T1	Bread, Milk, Butter
T2	Bread , Butter
T3	Bread, Milk
T4	Milk , Butter
T5	Bread

Frequent Pattern Growth Algorithm

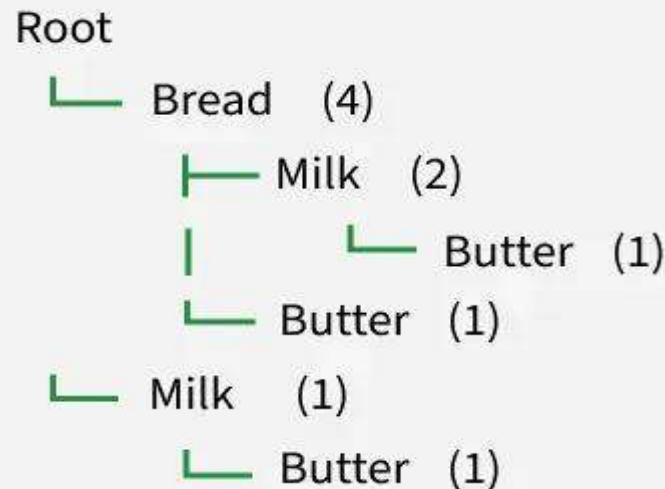
Step 3: Construct the FP-Tree.

Insert the sorted transactions one by one into the FP-tree, merging any shared prefixes.

FP-Tree Insertions:

- **From T1:** Bread → Milk → Butter
- **From T2:** Bread → Butter
- **From T3:** Bread → Milk
- **From T4:** Milk → Butter (new branch since it doesn't begin with Bread)
- **From T5:** Bread

Visual FP-Tree:



Frequent Pattern Growth Algorithm

Step 4: Determine Conditional Pattern Bases

A conditional pattern base contains all prefix paths leading to a specific item. Let's examine the paths ending with Butter.

Paths that end with Butter:

Bread → Milk → Butter (1 occurrence)

Bread → Butter (1 occurrence)

Milk → Butter (1 occurrence)

Thus, the conditional pattern base for Butter is:

[(Bread, Milk): 1, (Bread): 1, (Milk): 1]

Step 5: Build Conditional FP-Trees

Using the conditional pattern base, construct a smaller FP-tree for each item to identify frequent patterns involving that item. Butter's conditional FP-tree input:

- (Bread, Milk): 1

- (Bread): 1

- (Milk): 1

Count all items: Bread: 2 ,Milk: 2

Since both meet the support threshold (≥ 2), we can now generate frequent patterns:

- {Butter, Bread}

- {Butter, Milk}

- {Butter, Bread, Milk}

- Repeat the process for Milk and Bread as needed.**

Frequent Pattern Growth Algorithm

Step 6: Extract All Frequent Itemsets

From the FP-tree and conditional trees, we get these frequent itemsets:

- {Bread}
- {Milk}
- {Butter}
- {Bread, Milk}
- {Bread, Butter}
- {Milk, Butter}
- {Bread, Milk, Butter}

All of these appear at least 2 times in the transactions.

Summary Table

Frequent Itemset	Support Count
{Bread}	4
{Milk}	3
{Butter}	3
{Bread, Milk}	2
{Bread , Butter}	2
{Milk , Butter}	2
{Bread , Milk , Butter}	1 ✗ Not included (support < 2)

Frequent Pattern Growth Algorithm

Applications

- **Market Basket Analysis:** Identifying items that are frequently purchased together.
- **Recommendation Systems:** Suggesting products based on frequent item patterns.
- **Customer Behavior Analysis:** Understanding buying habits in retail and e-commerce.
- **Web Usage Mining:** Finding common browsing or click paths among users.
- **Text Mining:** Detecting frequently occurring words or phrases in documents.

Advantages

- **No Candidate Generation:** FP-Growth avoids generating candidate sets, improving efficiency.
- **Minimal Data Scans:** Only two database scans are required.
- **High Performance on Large Data:** Works very well for big and dense datasets.
- **Data Compression:** Uses an FP-tree to store transactions compactly.
- **Scalable and Fast:** Capable of mining complex and large frequent patterns quickly.

Limitations

- **High Memory Usage:** FP-tree can become large, consuming significant memory.
- **Complex Implementation:** Harder to understand and implement than Apriori.
- **Heavy Conditional Mining:** Building conditional FP-trees can be computationally costly.
- **Poor for Diverse Transactions:** Not ideal when transactions have many unique items.
- **Not Incremental:** Cannot easily update hence tree must be rebuilt when data changes.

Evaluation Metrics in Unsupervised Machine Learning

Evaluating unsupervised machine learning is challenging because there is no "ground truth" or labeled data to compare against. Instead, metrics focus on how well the algorithm discovers patterns, typically categorized into Internal Validation (measuring the structure of the clusters) and External Validation (comparing against known labels if available for testing). The main goal is to measure: Cluster quality, Similarity, Compactness, Separation, Pattern quality etc.

1. Internal Validation Metrics

These metrics use only the input data and the clustering results to measure quality

a) Silhouette Coefficient: Measures how similar a point is to its own cluster compared to other clusters. Scores range from -1 to 1; a high score indicates the object is well-matched to its own cluster and poorly matched to neighboring clusters.

$$s = \frac{b-a}{\max(a,b)}$$

Where: a = average distance within the same cluster

b = average distance to nearest cluster

Range

- **+1** → good clustering
- **0** → overlapping clusters
- **-1** → wrong clustering

Advantages

- Easy to interpret
- Works well for compact clusters

Disadvantages

- Expensive for large datasets

b) Davies-Bouldin Index: Calculates the average similarity between each cluster and its most similar one. A **lower score** is better, indicating that clusters are compact and well-separated from each other

$$DBI = \frac{1}{n} \sum_{i=1}^n \max_{j \neq i} \left(\frac{S_i + S_j}{M_{ij}} \right)$$

Evaluation Metrics in Unsupervised Machine Learning

Where:

- S_i = cluster scatter
- M_{ij} = distance between cluster centers

Interpretation

- **Lower DBI = Better clustering**

Advantages: Fast computation

Disadvantages: Sensitive to noise

c) Calinski-Harabasz Index (Variance Ratio Criterion): The ratio of the sum of between-cluster dispersion and within-cluster dispersion. Higher values generally indicate that clusters are dense and well-separated.

$$CH = \frac{Tr(B_k)/(k-1)}{Tr(W_k)/(n-k)}$$

Interpretation

- **Higher value = Better clustering**

Advantages: Efficient and popular

Disadvantages: Prefers convex clusters

d) Inertia (Within-Cluster Sum of Squares): Measures how internally coherent clusters are. It is commonly used in K-Means; however, lower inertia doesn't always mean better clustering, as it naturally decreases as you add more clusters

Evaluation Metrics in Unsupervised Machine Learning

2) External Validation Metrics

These are used when "true" labels are available for evaluation purposes (even if they weren't used for training).

(a) Rand Index (RI)

Measures agreement between predicted clusters and actual labels.

$$RI = \frac{TP+TN}{TP+FP+FN+TN} \quad \text{Range: 0 to 1, Higher is better}$$

b) Adjusted Rand Index (ARI): Computes similarity between two clusterings by considering all pairs of samples. It is "adjusted for chance," meaning a random labeling will yield a score close to 0. A score of 1 indicates a perfect match

Advantages

- More reliable than RI

Range: 1 → perfect clustering **0** → random clustering

- Negative → poor clustering

(c) Normalized Mutual Information (NMI): Measures the mutual information between the predicted clusters and the true labels, normalized to scale the results between 0 and 1.

Interpretation

- 0 → no mutual information

- 1 → perfect clustering

$$NMI(X, Y) = \frac{2I(X;Y)}{H(X)+H(Y)}$$

• Homogeneity, Completeness, and V-Measure:

- **Homogeneity:** Each cluster contains only members of a single class.
- **Completeness:** All members of a given class are assigned to the same cluster.
- **V-Measure:** The harmonic mean of homogeneity and completeness.

Evaluation Metrics in Unsupervised Machine Learning

3. Metrics for Association Rule Mining

Used in algorithms like:

- Apriori
- FP-Growth
- ECLAT

(a) Support: Frequency of occurrence of an itemset. $Support(A) = \frac{\text{Transactions containing } A}{\text{Total Transactions}}$

(b) Confidence: Probability that B occurs when A occurs $Confidence(A \rightarrow B) = \frac{Support(A \cup B)}{Support(A)}$

(c) Lift: Measures strength of association.

Interpretation

- Lift > 1 → positive association
- Lift = 1 → independent
- Lift < 1 → negative association

4. Metrics for Dimensionality Reduction

Used in: PCA, t-SNE, Autoencoders

(a) Reconstruction Error

Measures information loss after dimensionality reduction. Measures the difference between the original data and the data after it has been compressed (reduced) and then reconstructed. Lower error indicates better preservation of information.

$$Error = \|X - \hat{X}\|^2 \quad \text{Lower error means better representation.}$$

(b) Explained Variance Ratio (PCA)

Measures how much variance each principal component explains. Indicates the proportion of the dataset's total variance that is captured by each principal component.

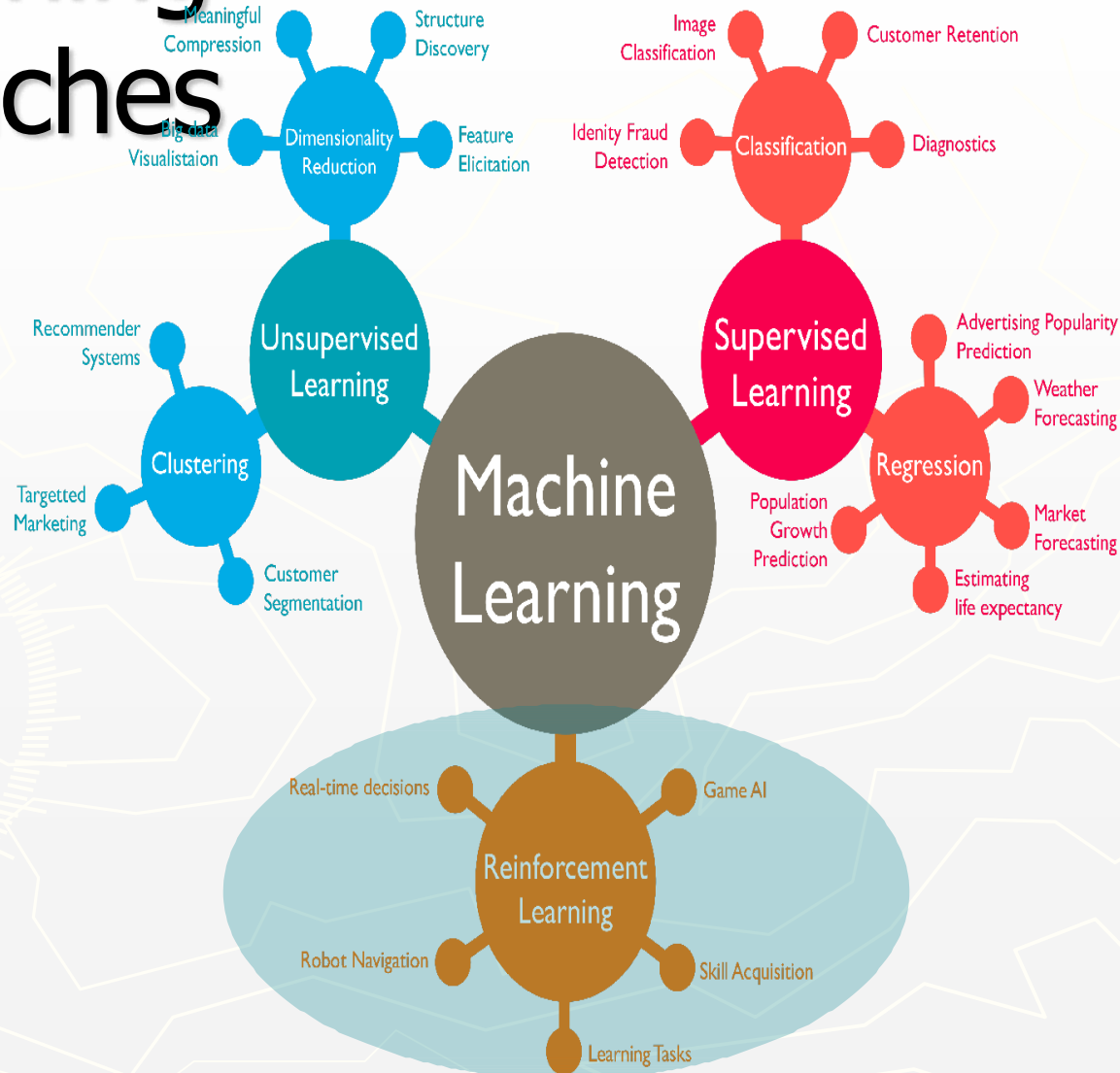
Where: λ_i = eigenvalue of component

$$EVR = \frac{\lambda_i}{\sum \lambda}$$

Evaluation Metrics in Unsupervised Machine Learning

Metric	Used For	Best Value
Silhouette Score	Clustering	Higher
Davies–Bouldin Index	Clustering	Lower
Calinski–Harabasz	Clustering	Higher
Rand Index	Cluster comparison	Higher
ARI	Cluster comparison	Higher
NMI	Cluster comparison	Higher
Support	Association rules	Higher
Confidence	Association rules	Higher
Lift	Association rules	> 1
Reconstruction Error	PCA/Autoencoder	Lower
Explained Variance Ratio	PCA	Higher

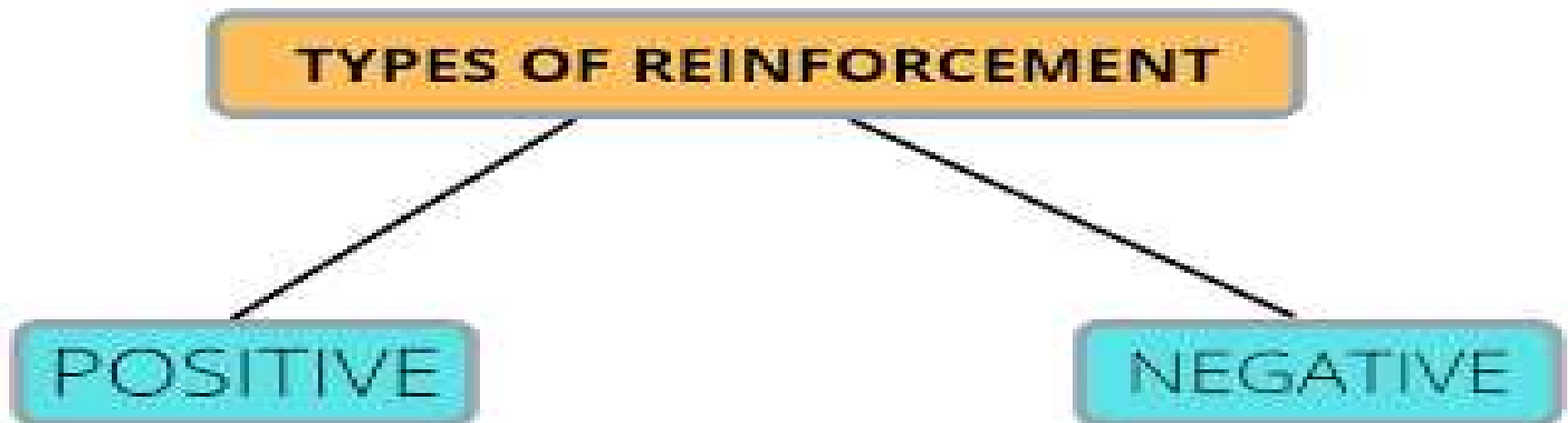
Machine Learning Branches



Machine Learning Branches: Reinforcement Learning

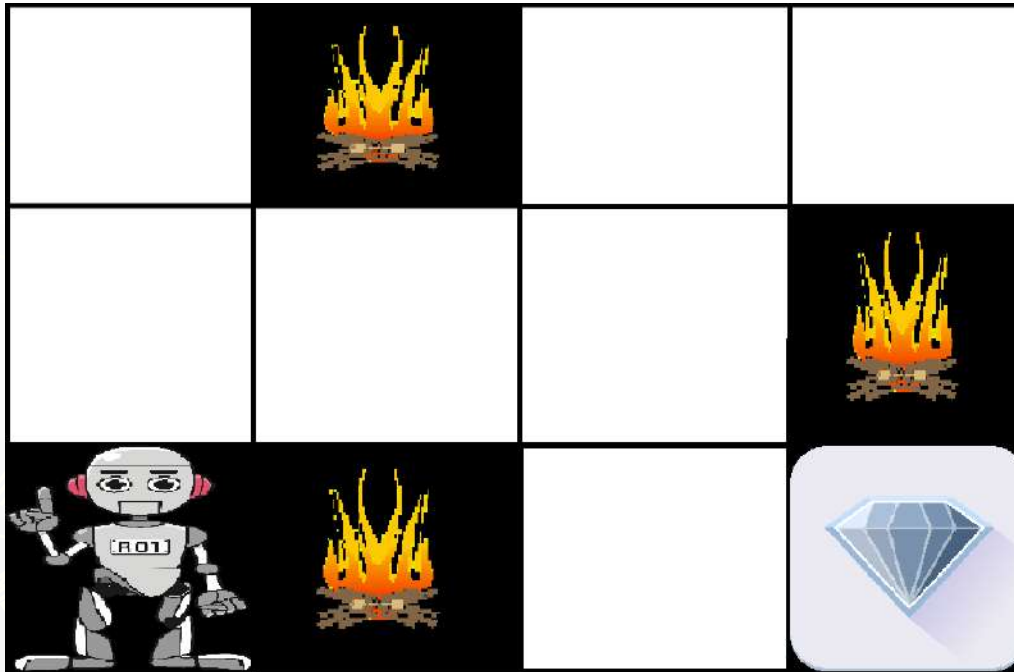
Reinforcement learning works on a feedback-based process, in which an AI agent (A software component) automatically explore its surrounding by hitting & trail, taking action, learning from experiences, and improving its performance. Agent gets rewarded for each good action and get punished for each bad action; hence the goal of reinforcement learning agent is to maximize the rewards. In Reinforcement Learning, the agent learns automatically using feedbacks without any labeled data, unlike supervised learning.

Due to its way of working, reinforcement learning is employed in different fields such as **Game theory, Operation Research, Information theory, multi-agent systems.**



Reinforcement Learning: Example

The agent is supposed to find the best possible path to reach the reward. The following problem explains the problem more easily.



The above image shows the robot, diamond, and fire. The goal of the robot is to get the reward that is the diamond and avoid the hurdles that are fire. The robot learns by trying all the possible paths and then choosing the path which gives him the reward with the least hurdles. Each right step will give the robot a reward and each wrong step will subtract the reward of the robot. The total reward will be calculated when it reaches the final reward that is the diamond. The problem is as follows: We have an agent and a reward, with many hurdles in between.

Reinforcement Learning: Example

Main points in Reinforcement learning –

- **Input:** The input should be an initial state from which the model will start
- **Output:** There are many possible outputs as there are a variety of solutions to a particular problem
- **Training:** The training is based upon the input, The model will return a state and the user will decide to reward or punish the model based on its output.
- The model keeps continues to learn.
- The best solution is decided based on the maximum reward.
- **Difference between Reinforcement learning and Supervised learning:**

Reinforcement learning	Supervised learning
Reinforcement learning is all about making decisions sequentially. In simple words, we can say that the output depends on the state of the current input and the next input depends on the output of the previous input	In Supervised learning, the decision is made on the initial input or the input given at the start
In Reinforcement learning decision is dependent, So we give labels to sequences of dependent decisions	In supervised learning the decisions are independent of each other so labels are given to each decision.
Example: Chess game, text summarization	Example: Object recognition, spam detection

Reinforcement Learning

Terms used in Reinforcement Learning

- **Agent():** An entity that can perceive/explore the environment and act upon it.
- **Environment():** A situation in which an agent is present or surrounded by. In RL, we assume the stochastic environment, which means it is random in nature.
- **Action():** Actions are the moves taken by an agent within the environment.
- **State():** State is a situation returned by the environment after each action taken by the agent.
- **Reward():** A feedback returned to the agent from the environment to evaluate the action of the agent.
- **Policy():** Policy is a strategy applied by the agent for the next action based on the current state.
- **Value():** It is expected long-term return with the discount factor and opposite to the short-term reward.

Reinforcement Learning in ML



Reinforcement Learning

Key Features of Reinforcement Learning

- In RL, the agent is not instructed about the environment and what actions need to be taken.
- It is based on the hit and trial process.
- The agent takes the next action and changes states according to the feedback of the previous action.
- The agent may get a delayed reward.
- The environment is stochastic, and the agent needs to explore it to reach to get the maximum positive rewards.

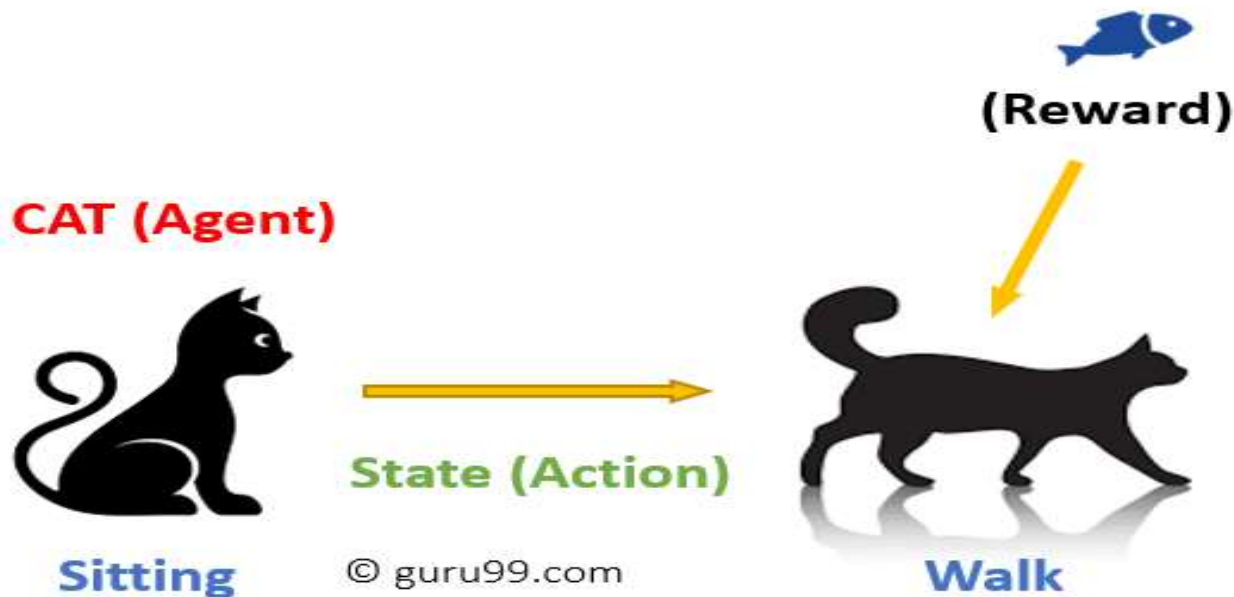
Approaches to implement Reinforcement Learning

- There are mainly three ways to implement reinforcement-learning in ML, which are:
- **Value-based:** The value-based approach is about to find the optimal value function, which is the maximum value at a state under any policy. Therefore, the agent expects the long-term return at any state(s) under policy π .
- **Policy-based:** Policy-based approach is to find the optimal policy for the maximum future rewards without using the value function. In this approach, the agent tries to apply such a policy that the action performed in each step helps to maximize the future reward.
- The policy-based approach has mainly two types of policy:
 - Deterministic:** The same action is produced by the policy (π) at any state.
 - Stochastic:** In this policy, probability determines the produced action.
- **Model-based:** In the model-based approach, a virtual model is created for the environment, and the agent explores that environment to learn it. There is no particular solution or algorithm for this approach because the model representation is different for each environment.

Reinforcement Learning: Positive

Positive Reinforcement Learning: Positive reinforcement learning specifies increasing the tendency that the required behaviour would occur again by adding something. It enhances the strength of the behaviour of the agent and positively impacts it. This type of reinforcement can sustain the changes for a long time, but too much positive reinforcement may lead to an overload of states that can reduce the consequences.

House (environment)



Reinforcement Learning: Negative

Negative Reinforcement Learning: Negative reinforcement learning works exactly opposite to the positive RL. It increases the tendency that the specific behavior would occur again by avoiding the negative condition. It can be more effective than the positive reinforcement depending on situation and behavior, but it provides reinforcement only to meet minimum behavior.

Negative Reinforcement Learning



Behavior Strengthened by Removing an Unpleasant Stimulus

Reinforcement Learning

Core Components

1. Policy

- Defines the agent's behavior i.e maps states for actions.
- Can be simple rules or complex computations.
- **Example:** An autonomous car maps pedestrian detection to make necessary stops.

2. Reward Signal

- Represents the goal of the RL problem.
- Guides the agent by providing feedback (positive/negative rewards).
- **Example:** For self-driving cars rewards can be fewer collisions, shorter travel time, lane discipline.

3. Value Function

- Evaluates long-term benefits, not just immediate rewards.
- Measures desirability of a state considering future outcomes.
- **Example:** A vehicle may avoid reckless maneuvers (short-term gain) to maximize overall safety and efficiency.

4. Model

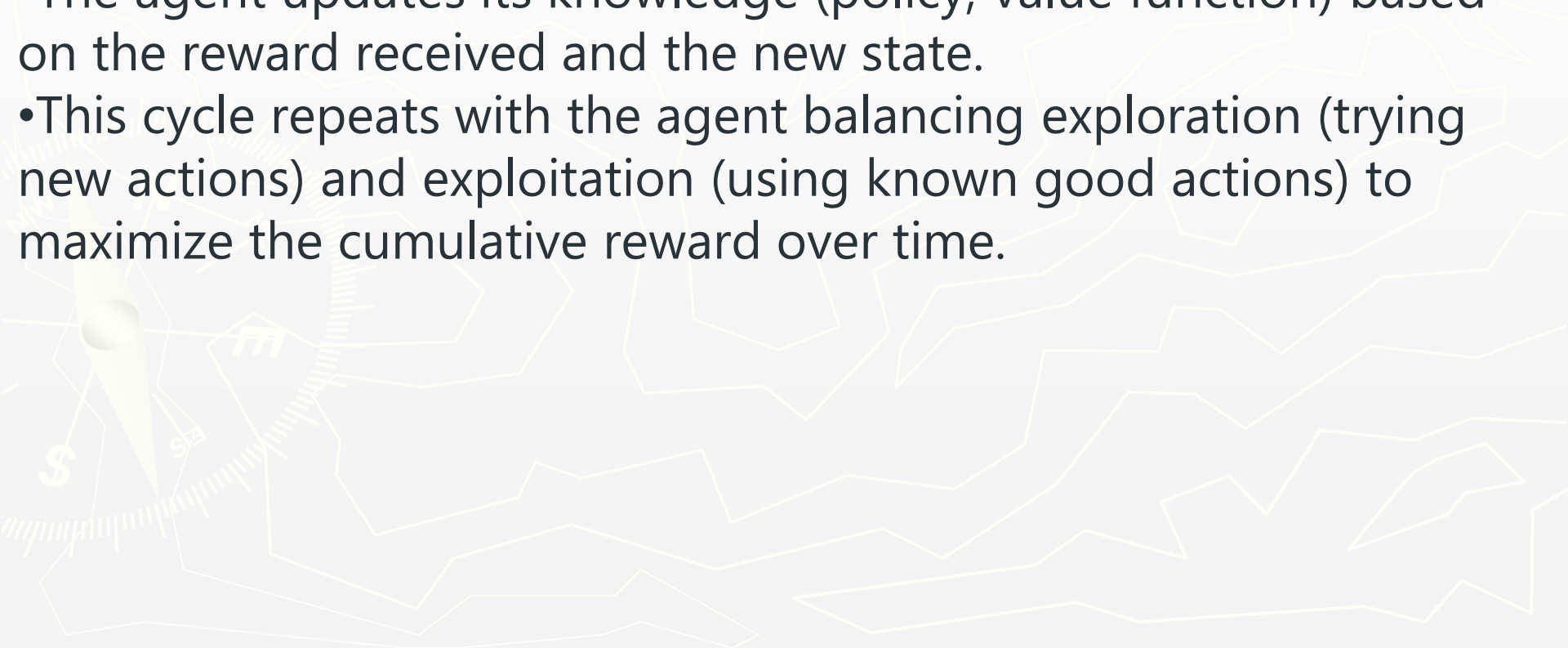
- Simulates the environment to predict outcomes of actions.
- Enables planning and foresight.
- **Example:** Predicting other vehicles' movements to plan safer routes.

Reinforcement Learning

Working of Reinforcement Learning

The agent interacts iteratively with its environment in a feedback loop:

- The agent observes the current state of the environment.
- It chooses and performs an action based on its policy.
- The environment responds by transitioning to a new state and providing a reward (or penalty).
- The agent updates its knowledge (policy, value function) based on the reward received and the new state.
- This cycle repeats with the agent balancing exploration (trying new actions) and exploitation (using known good actions) to maximize the cumulative reward over time.



Reinforcement Learning

Elements of Reinforcement Learning

There are four main elements of Reinforcement Learning, which are given below:

1. Policy

2. Reward Signal

3. Value Function

4. Model of the environment

1) Policy: A policy can be defined as a way how an agent behaves at a given time. It maps the perceived states of the environment to the actions taken on those states. A policy is the core element of the RL as it alone can define the behavior of the agent. In some cases, it may be a simple function or a lookup table, whereas, for other cases, it may involve general computation as a search process. It could be deterministic or a stochastic policy:

For deterministic policy: $a = \pi(s)$

For stochastic policy: $\pi(a | s) = P[A_t = a | S_t = s]$

2) Reward Signal: The goal of reinforcement learning is defined by the reward signal. At each state, the environment sends an immediate signal to the learning agent, and this signal is known as a reward signal. These rewards are given according to the good and bad actions taken by the agent. The agent's main objective is to maximize the total number of rewards for good actions. The reward signal can change the policy, such as if an action selected by the agent leads to low reward, then the policy may change to select other actions in the future.

3) Value Function: The value function gives information about how good the situation and action are and how much reward an agent can expect. A reward indicates the immediate signal for each good and bad action, whereas a value function specifies the good state and action for the future. The value function depends on the reward as, without reward, there could be no value. The goal of estimating values is to achieve more rewards.

4) Model: The last element of reinforcement learning is the model, which mimics the behavior of the environment. With the help of the model, one can make inferences about how the environment will behave. Such as, if a state and an action are given, then a model can predict the next state

Reinforcement Learning

The Bellman Equation

It will be a difficult condition for the agent whether he should go up or down as each block has the same value. So, the brutforce approach is not suitable for the agent to reach the destination. Hence to solve the problem, we will use the **Bellman equation**, which is the main concept behind reinforcement learning. The Bellman equation was introduced by the Mathematician **Richard Ernest Bellman in the year 1953**, and hence it is called as a Bellman equation. It is associated with dynamic programming and used to calculate the values of a decision problem at a certain point by including the values of previous states. It is a way of calculating the value functions in dynamic programming or environment that leads to modern reinforcement learning. The key-elements used in Bellman equations are:

- Action performed by the agent is referred to as "a"
- State occurred by performing the action is "s."
- The reward/feedback obtained for each good and bad action is "R."
- A discount factor is Gamma " γ ."

The **Bellman equation** can be written as:

$$V(s) = \max [R(s,a) + \gamma V(s')]]$$

Where,

$V(s)$ = value calculated at a particular point.

$R(s,a)$ = Reward at a particular state s by performing an action.

γ = Discount factor

$V(s')$ = The value at the previous state.

In the above equation, we are taking the max of the complete values because the agent tries to find the optimal solution always. So now, using the Bellman equation, we will find value at each state of the given environment. We will start from the block, which is next to the target block.

Reinforcement Learning

How to represent the agent state?

We can represent the agent state using the **Markov State** that contains all the required information from the history. The State S_t is Markov state if it follows the given condition:

$$[S_{t+1} | S_t] = P[S_{t+1} | S_1, \dots, S_t]$$

The Markov state follows the **Markov property**, which says that the future is independent of the past and can only be defined with the present. The RL works on fully observable environments, where the agent can observe the environment and act for the new state. The complete process is known as Markov Decision process, which is explained below:

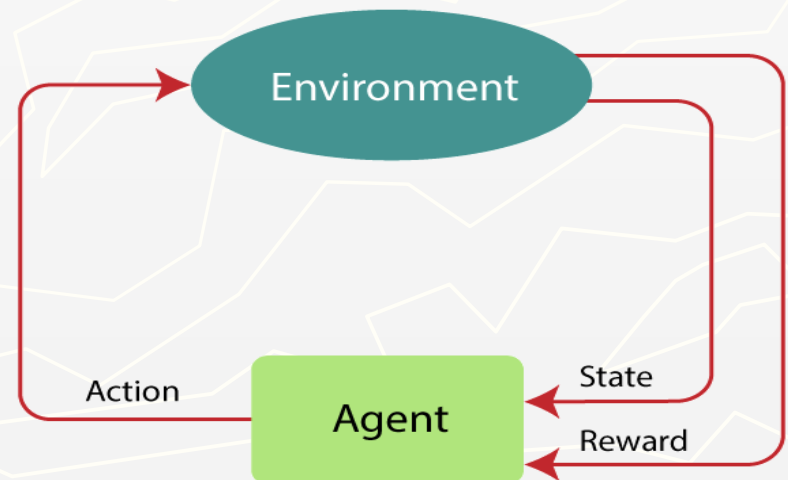
Markov Decision Process

Markov Decision Process or MDP, is used to **formalize the reinforcement learning problems**. If the environment is completely observable, then its dynamic can be modeled as a **Markov Process**. In MDP, the agent constantly interacts with the environment and performs actions; at each action, the environment responds and generates a new state.

MDP is used to describe the environment for the RL, and almost all the RL problem can be formalized using MDP.

MDP contains a tuple of four elements (S, A, P_a, R_a) :

- A set of finite States S
- A set of finite Actions A
- Rewards received after transitioning from state S to state S' , due to action a .
- Probability P_a .



Reinforcement Learning

MDP uses **Markov property**, and to better understand the MDP, we need to learn about it.

Markov Property:

It says that **"If the agent is present in the current state S_1 , performs an action a_1 and move to the state s_2 , then the state transition from s_1 to s_2 only depends on the current state and future action and states do not depend on past actions, rewards, or states."**

Or, in other words, as per Markov Property, the current state transition does not depend on any past action or state. Hence, MDP is an RL problem that satisfies the Markov property. Such as in a **Chess game, the players only focus on the current state and do not need to remember past actions or states.**

Finite MDP:

A finite MDP is when there are finite states, finite rewards, and finite actions. In RL, we consider only the finite MDP.

Markov Process:

Markov Process is a memoryless process with a sequence of random states $S_1, S_2, \dots, S_t$ that uses the Markov Property. Markov process is also known as Markov chain, which is a tuple (S, P) on state S and transition function P . These two components (S and P) can define the dynamics of the system.

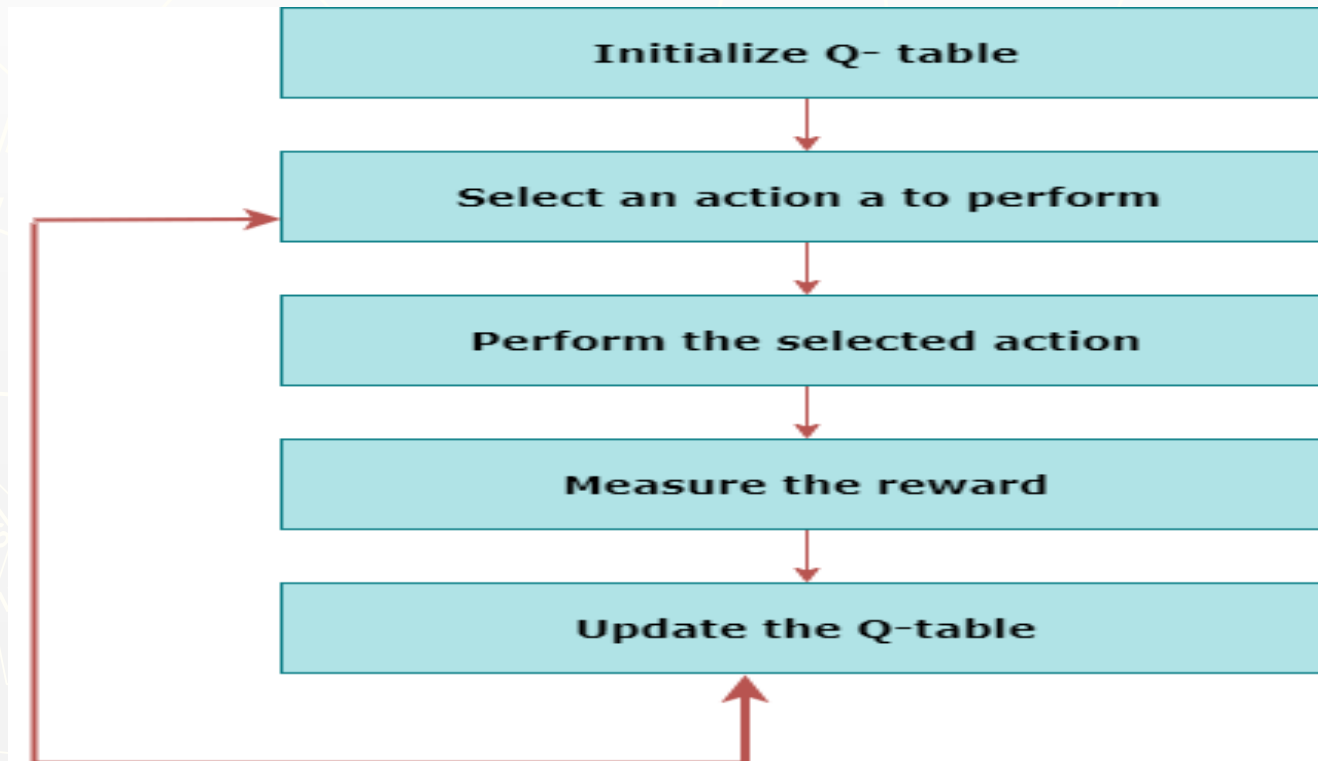
Reinforcement Learning

Reinforcement Learning Algorithms

Reinforcement learning algorithms are mainly used in AI applications and gaming applications. The main used algorithms are:

•Q-Learning:

- Q-learning is an **Off policy RL algorithm**, which is used for the temporal difference Learning. The temporal difference learning methods are the way of comparing temporally successive predictions.
- It learns the value function $Q(S, a)$, which means how good to take action "**a**" at a particular state "**s**."
- The below flowchart explains the working of Q- learning:



Reinforcement Learning

Reinforcement Learning Algorithms

•State Action Reward State action (SARSA):

- SARSA stands for **State Action Reward State action**, which is an **on-policy** temporal difference learning method. The on-policy control method selects the action for each state while learning using a specific policy.
- The goal of SARSA is to calculate the **$Q \pi (s, a)$ for the selected current policy π and all pairs of (s-a).**
- The main difference between Q-learning and SARSA algorithms is that **unlike Q-learning, the maximum reward for the next state is not required for updating the Q-value in the table.**
- In SARSA, new action and reward are selected using the same policy, which has determined the original action.
- The SARSA is named because it uses the quintuple **$Q(s, a, r, s', a')$** . Where,
s: original state
a: Original action
r: reward observed while following the states
s' and a': New state, action pair.

•Deep Q Neural Network (DQN):

- As the name suggests, DQN is a **Q-learning using Neural networks**.
- For a big state space environment, it will be a challenging and complex task to define and update a Q-table.
- To solve such an issue, we can use a DQN algorithm. Where, instead of defining a Q-table, neural network approximates the Q-values for each action and state.

Reinforcement Learning

Problem

A robot moves in the following grid to reach the goal.

S			
	X		
			G

Where:

- S = Start
- G = Goal
- X= Obstacle

Given

Actions

$A = \{\text{Up, Down, Left, Right}\}$

Rewards

Parameters

Learning rate: $\alpha=0.5$ Discount factor: $\gamma=0.9$

Condition	Reward
Reach Goal	+100
Hit Obstacle	-100
Normal Move	-1

Reinforcement Learning

Step 1: Initialize Q-Table

Initially all Q-values are 0.

State	Up	Down	Left	Right
S	0	0	0	0

Step 2: Apply Q-Learning Formula

$$Q(s, a) = Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Step 3: Episode 1

Robot starts from: $S=(0,0)$

Suppose robot chooses: Right and moves to: $(0,1)$

Reward received: $r=-1$

Step 4: Update Q-Value

Current Q-value: $Q(S, \text{Right})=0$

Future maximum Q-value: $\max Q((0,1), a)=0$

Substitute into formula: $Q(S, \text{Right}) = 0 + 0.5[-1 + 0.9(0) - 0]$

Result: $Q(S, \text{Right})=-0.5$

Step 5: Next Move

Robot moves from: $(0,1) \rightarrow (0,2)$

Reward: -1 Update: $Q((0,1), \text{Right}) = 0 + 0.5[-1 + 0.9(0) - 0]$

Result: $Q((0,1), \text{Right})=-0.5$

Reinforcement Learning

Step 6: Reaching Goal

Suppose robot reaches goal from state: $(2,2) \rightarrow (2,3)$

Reward: +100

Update equation:

$$Q((2,2), \text{Right}) = 0 + 0.5[100 + 0.9(0) - 0]$$

Result: $Q((2,2), \text{Right}) = 50$

Step 7: Updated Q-Table

State	Best Action	Q-value
(0,0)	Right	-0.5
(0,1)	Right	-0.5
(2,2)	Right	50

Optimal Path Learned

S $\rightarrow$ $\rightarrow$ $\rightarrow$

$\downarrow$
 $\downarrow$
G

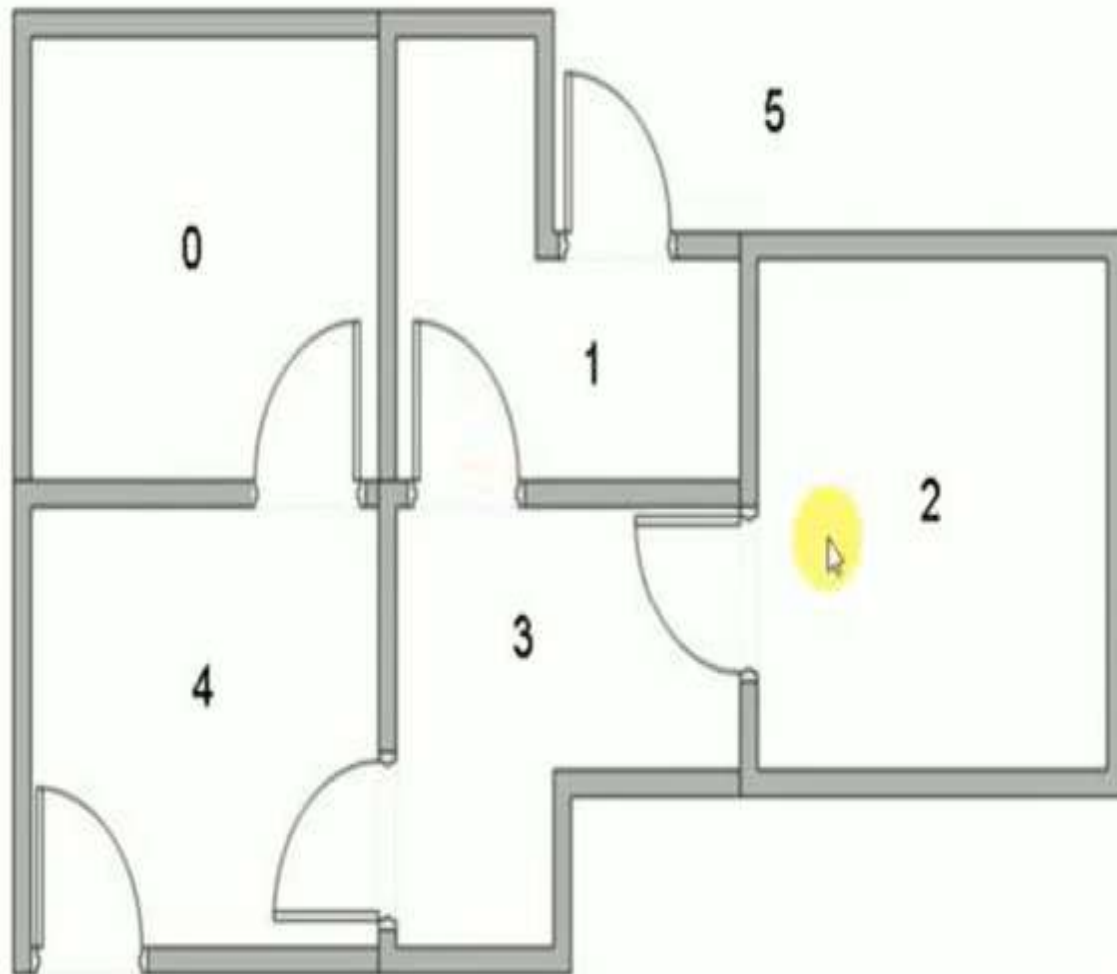
Step 8: After Many Episodes

After repeated learning:

State	Optimal Action
(0,0)	Right
(0,1)	Right
(0,2)	Down
(1,2)	Down
(2,2)	Right

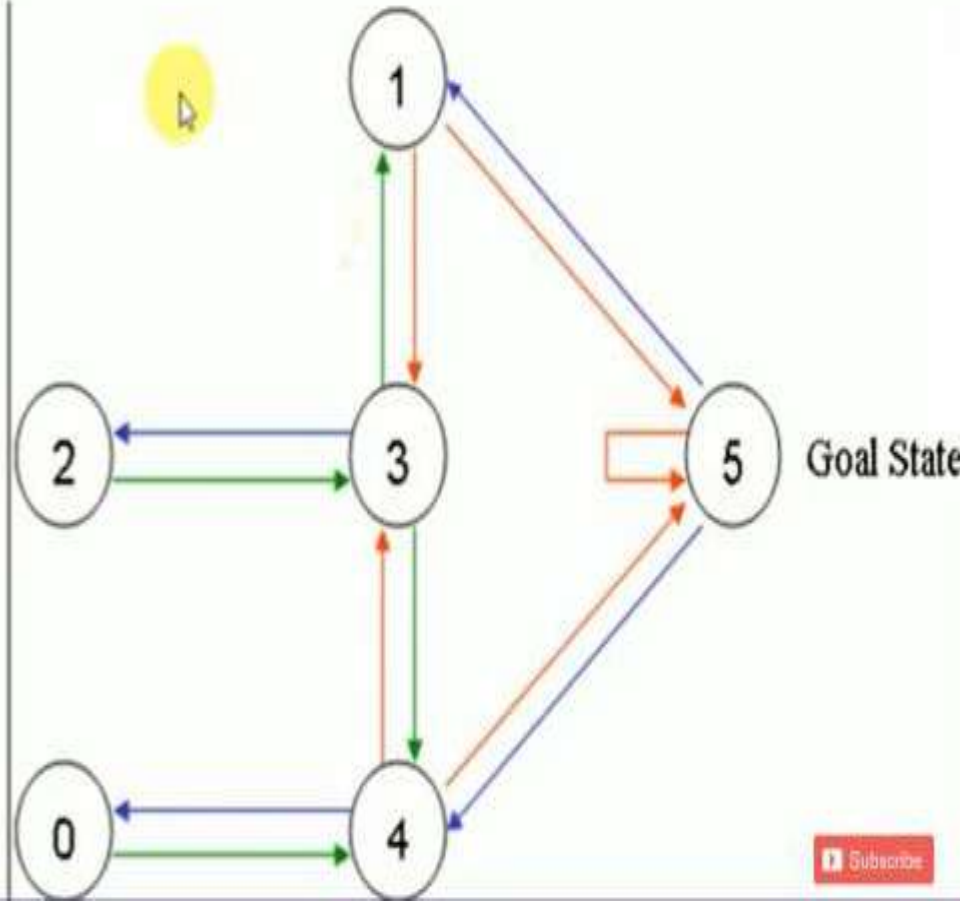
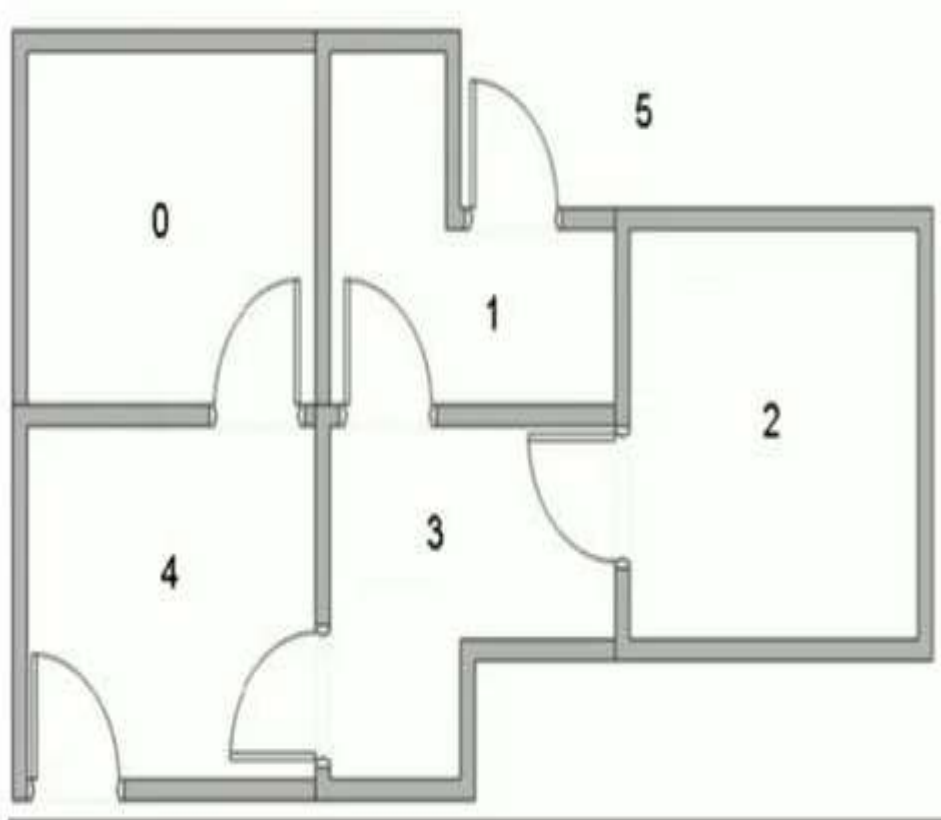
Reinforcement Learning(Q-learning):Example

- Suppose we have 5 rooms in a building connected by doors as shown in the figure below. We'll number each room 0 through 4. The outside of the building can be thought of as one big room (5). Notice that doors 1 and 4 lead into the building from room 5 (outside).



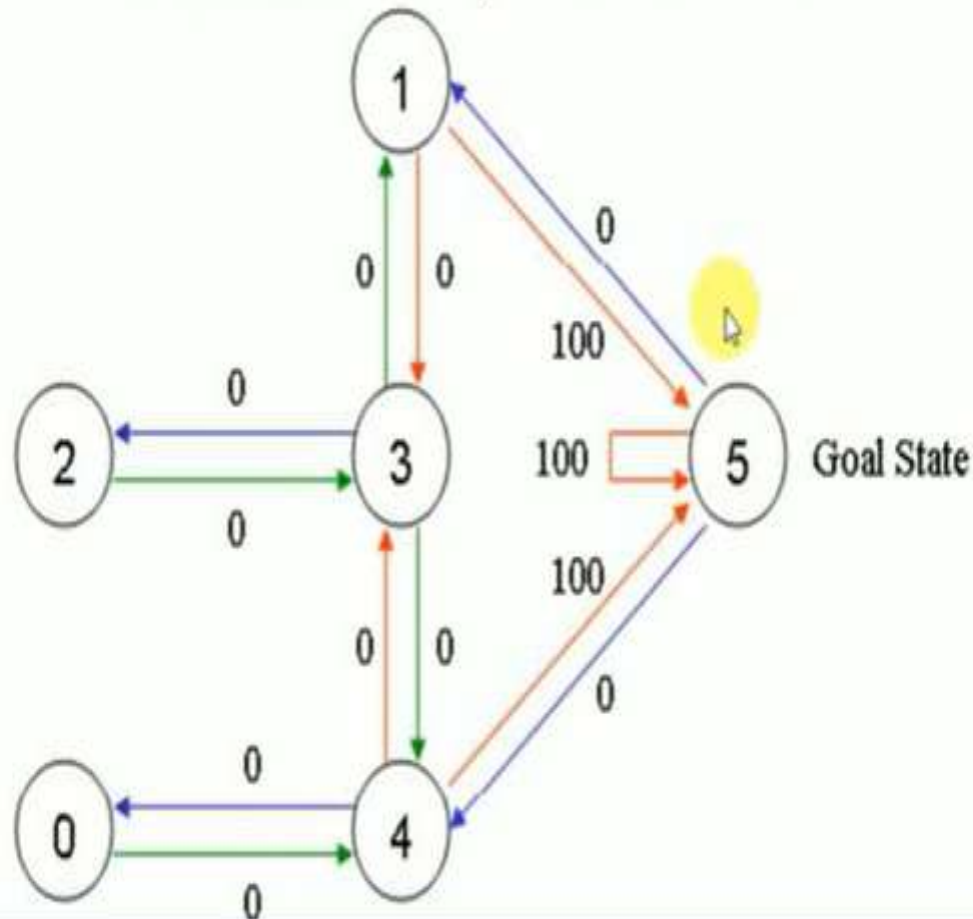
Reinforcement Learning(Q-learning):Example

- We can represent the rooms on a graph, each room as a node, and each door as a link.



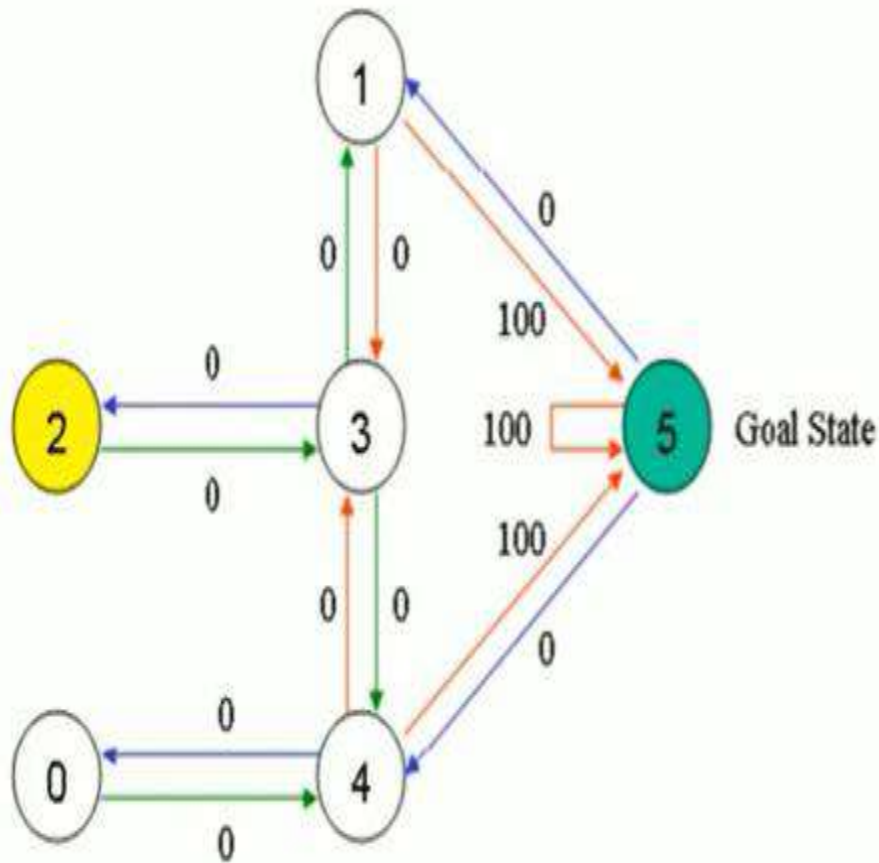
Reinforcement Learning(Q-learning):Example

- The goal room is number 5
- The doors that lead immediately to the goal have an instant reward of 100. Other doors not directly connected to the target room have zero reward.
- Each arrow contains an instant reward value, as shown below:



Reinforcement Learning(Q-learning):Example

- We can put the state diagram and the instant reward values into the following reward table, "matrix R". The -1's in the table represent null values (i.e.; where there isn't a link between nodes). For example, State 0 cannot go to State 1.



State	Action					
	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

Reinforcement Learning(Q-learning):Example

Q learning algorithm

For each s, a initialize the table entry $\hat{Q}(s, a)$ to zero.

Observe the current state s

Do forever:

- Select an action a and execute it
- Receive immediate reward r
- Observe the new state s'
- Update the table entry for $\hat{Q}(s, a)$ as follows:

$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a')$$

- $s \leftarrow s'$

Reinforcement Learning(Q-learning):Example

- Learning rate = 0.8 and the initial state as Room 1.
- Initialize matrix Q as a zero matrix:

$$R = \begin{matrix} & \text{Action} \\ \text{State} & 0 & 1 & 2 & 3 & 4 & 5 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{matrix}$$

$$Q = \begin{matrix} & 0 & 1 & 2 & 3 & 4 & 5 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

Reinforcement Learning(Q-learning):Example

- Look at the second row (state 1) of matrix R.
- There are two possible actions for the current state 1: go to state 3, or go to state 5.
- By random selection, we select to go to 5 as our action.

		Action					
State		0	1	2	3	4	5
0		-1	-1	-1	-1	0	-1
1		-1	-1	-1	0	-1	100
2		-1	-1	-1	0	-1	-1
3		-1	0	0	-1	0	-1
4		0	-1	-1	0	-1	100
5		-1	0	-1	-1	0	100

Reinforcement Learning(Q-learning):Example

- Now let's imagine what would happen if our agent were in state 5 (next state).
- Look at the sixth row of the reward matrix R (i.e. state 5).
- It has 3 possible actions: go to state 1, 4 or 5.
- $Q(\text{state}, \text{action}) = R(\text{state}, \text{action}) + \text{Gamma} * \text{Max}[Q(\text{next state}, \text{all actions})]$
- $Q(1, 5) = R(1, 5) + 0.8 * \text{Max}[Q(5, 1), Q(5, 4), Q(5, 5)] = 100 + 0.8 * 0 = 100$

State	Action					
	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

		0	1	2	3	4	5
$Q =$	0	0	0	0	0	0	0
	1	0	0	0	0	0	0
	2	0	0	0	0	0	0
	3	0	0	0	0	0	0
	4	0	0	0	0	0	0
	5	0	0	0	0	0	0

Reinforcement Learning(Q-learning):Example

- The next state, 5, now becomes the current state.
- Because 5 is the goal state, we've finished one episode.
- Our agent's brain now contains an updated matrix Q as:

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

Reinforcement Learning(Q-learning):Example

- For the next episode, we randomly choose the initial state – say 3 (can go to 1, 2 & 4)

$$R = \begin{array}{c|cccccc} & \text{Action} \\ \text{State} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & -1 & -1 & -1 & -1 & 0 & -1 \\ 1 & -1 & -1 & -1 & 0 & -1 & 100 \\ 2 & -1 & -1 & -1 & 0 & -1 & -1 \\ 3 & -1 & 0 & 0 & -1 & 0 & -1 \\ 4 & 0 & -1 & -1 & 0 & -1 & 100 \\ 5 & -1 & 0 & -1 & -1 & 0 & 100 \end{array}$$

- Now we imagine that we are in state 1 (next state).

Reinforcement Learning(Q-learning):Example

- Now we imagine that we are in state 1 (next state).
- Look at the second row of reward matrix R (i.e. state 1).
- It has 2 possible actions: go to state 3 or state 5.
- Then, we compute the Q value:
- $Q(\text{state}, \text{action}) = R(\text{state}, \text{action}) + \text{Gamma} * \text{Max}[Q(\text{next state}, \text{all actions})]$
- $Q(3, 1) = R(3, 1) + 0.8 * \text{Max}[Q(1, 3), Q(1, 5)] = 0 + 0.8 * \text{Max}(0, 100) = 80$

$$R = \begin{array}{c|cccccc} & \text{Action} \\ \text{State} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & -1 & -1 & -1 & -1 & 0 & -1 \\ 1 & -1 & -1 & -1 & 0 & -1 & 100 \\ 2 & -1 & -1 & -1 & 0 & -1 & -1 \\ 3 & -1 & 0 & 0 & -1 & 0 & -1 \\ 4 & 0 & -1 & -1 & 0 & -1 & 100 \\ 5 & -1 & 0 & -1 & -1 & 0 & 100 \end{array}$$

$$Q = \begin{array}{c|cccccc} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 100 \\ 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$

Reinforcement Learning(Q-learning):Example

- Now we imagine that we are in state 1 (next state).
- Look at the second row of reward matrix R (i.e. state 1).
- It has 2 possible actions: go to state 3 or state 5.
- Then, we compute the Q value:
- $Q(\text{state}, \text{action}) = R(\text{state}, \text{action}) + \text{Gamma} * \text{Max}[Q(\text{next state}, \text{all actions})]$
- $Q(3, 1) = R(3, 1) + 0.8 * \text{Max}[Q(1, 3), Q(1, 5)] = 0 + 0.8 * \text{Max}(0, 100) = 80$

State	Action					
	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

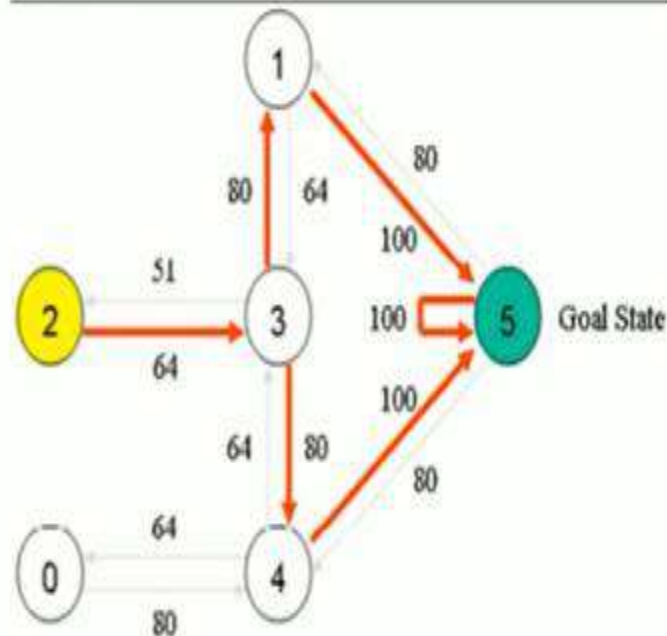
	Action					
	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	0
2	0	0	0	0	0	100
3	0	80	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	0	0	0

Reinforcement Learning(Q-learning):Example

- If our agent learns more through further episodes, it will finally reach convergence values in matrix Q like:

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 64 & 0 & 0 \\ 0 & 80 & 51 & 0 & 80 & 0 \\ 64 & 0 & 0 & 64 & 0 & 100 \\ 0 & 80 & 0 & 0 & 80 & 100 \end{bmatrix} \end{matrix}$$

- Tracing the best sequences of states is as simple as following the links with the highest values at each state.

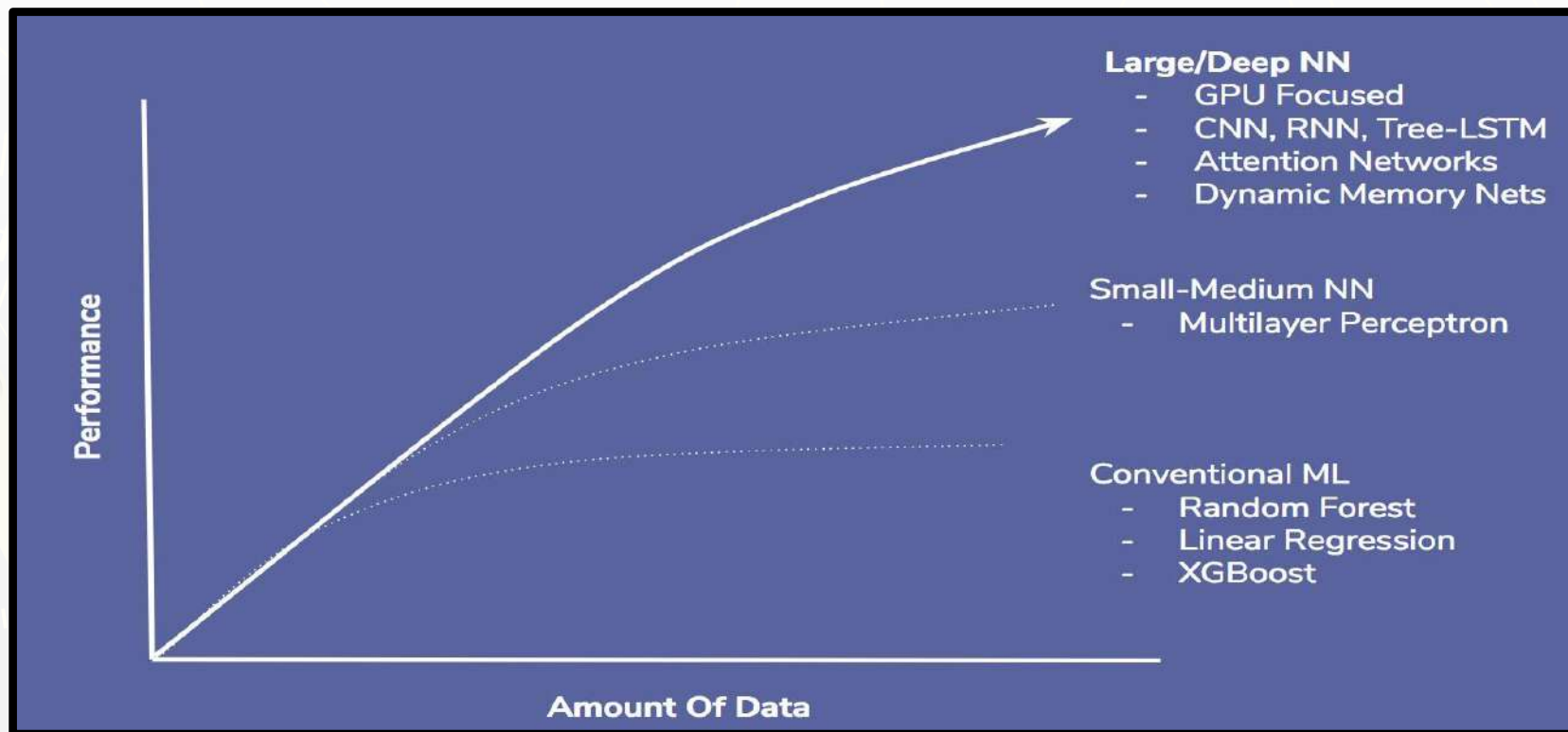


Part II: Deep Learning

Deep learning (DL) is the type of machine learning (ML) that resembles human brains, where it learns from data by using artificial neural networks. Just like human brains, these deep neural networks learn from real-life examples. In a few cases, it has surpassed human intelligence, just like Google's AlphaGo has defeated number one Go Player Ke Jie.

Why now? In recent years two things became available:

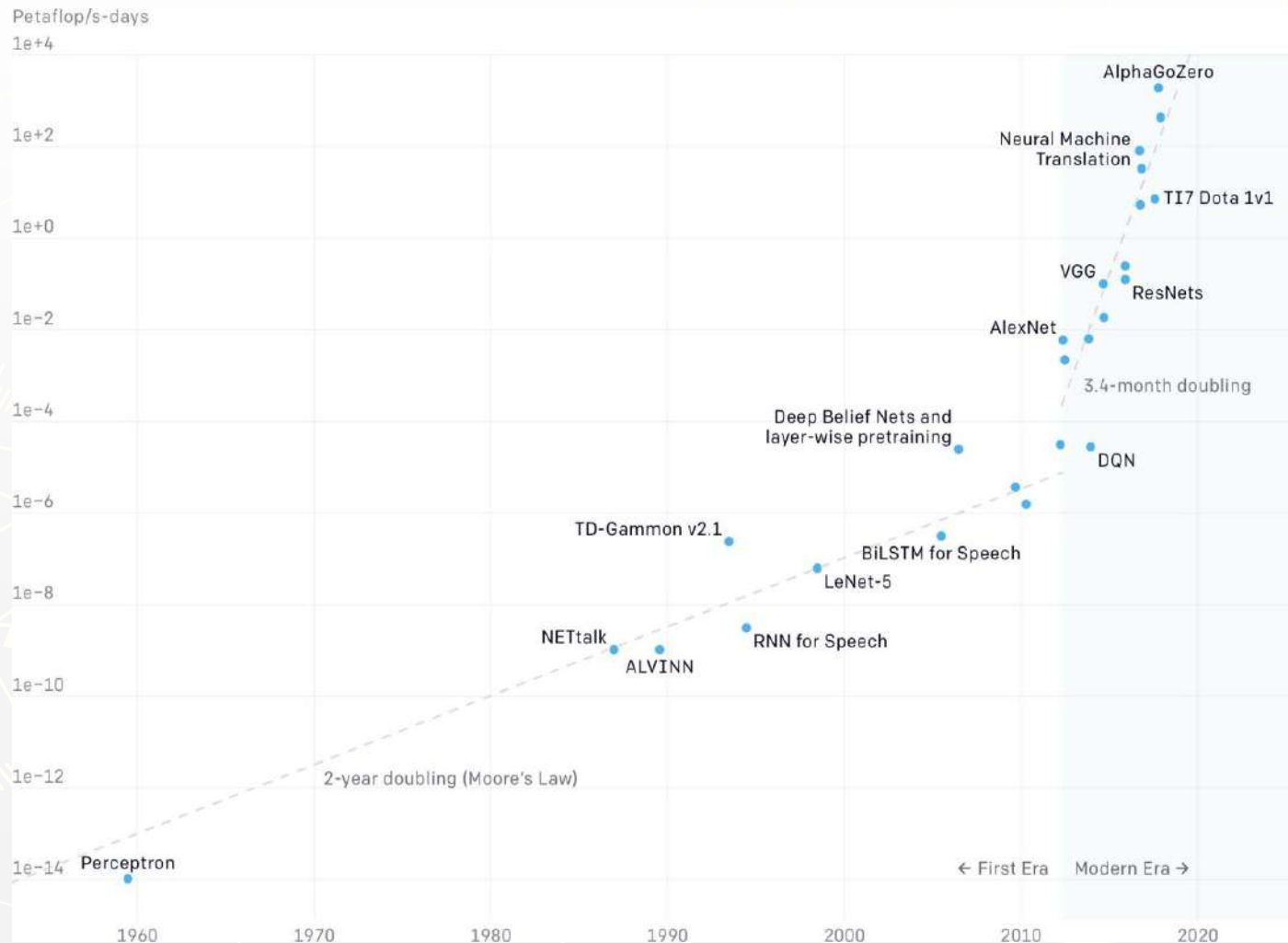
1. A lot of data



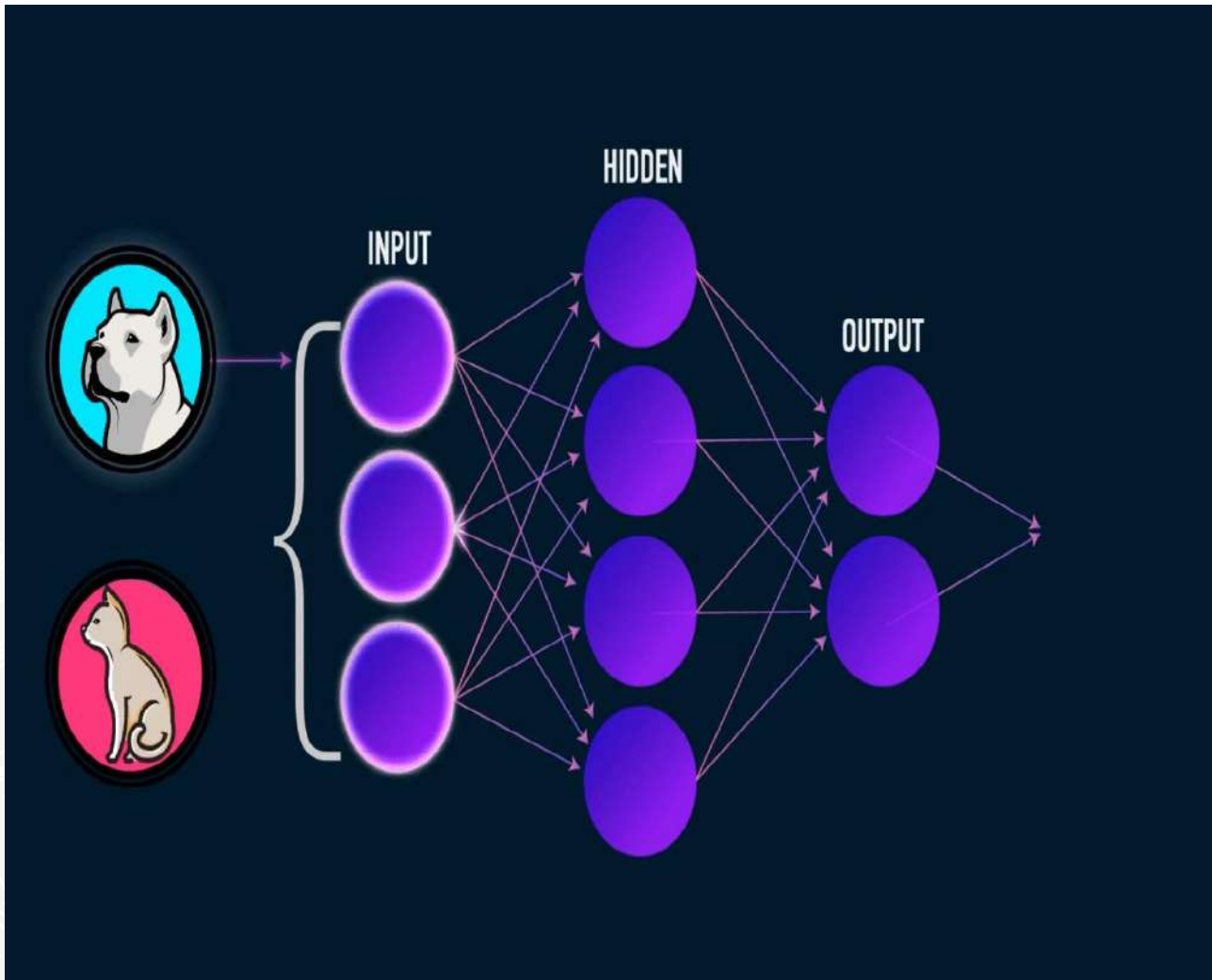
Part II: Deep Learning

Why now? In recent years two things became available:

2. *Necessary compute*



What is deep learning?

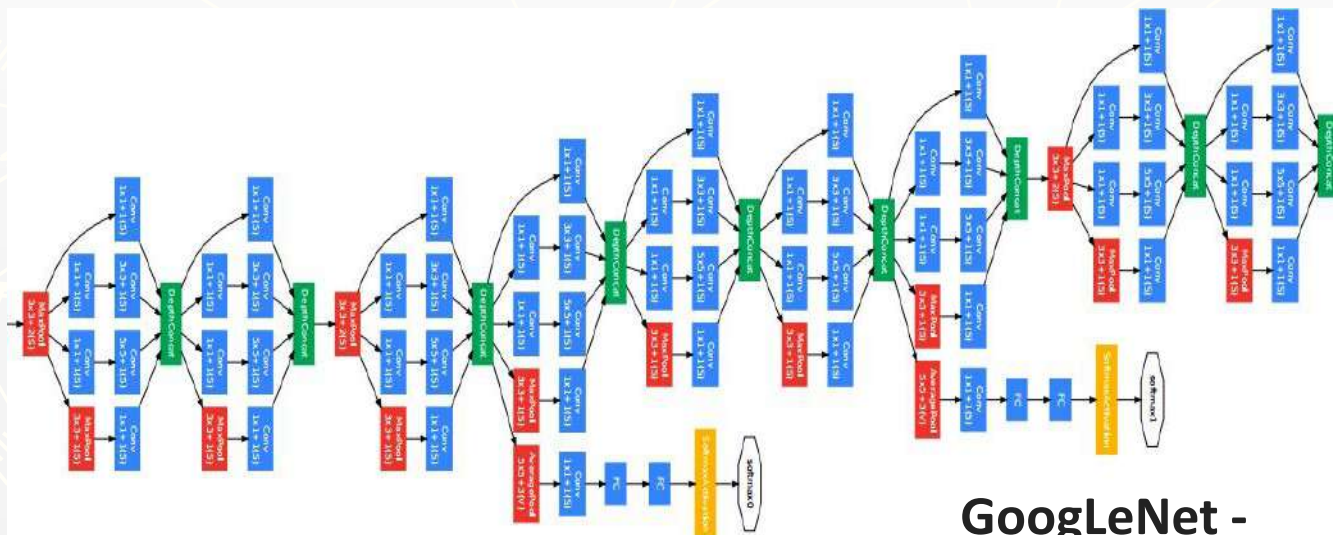


Rosenblatt -
1961

What is new in deep learning?

What is new (among other things) is a learning algorithm called **backpropagation** which allows to train **deep neural nets**.

State-of-the-art networks can have **over 200 layers!**

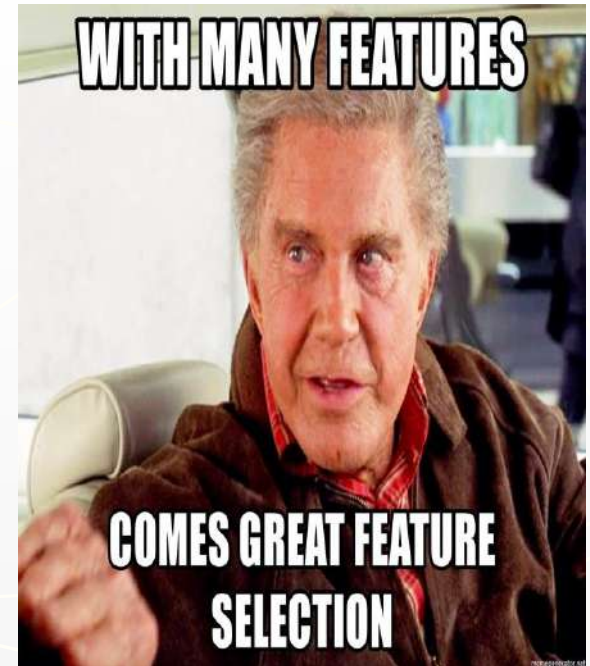


Difference classical ML vs. Deep Learning

Classical ML methods don't handle high dimensionality well.

⇒ **dimensionality reduction** & **feature selection**

Deep neural nets learn **compact representations** of data even in a high dimensionality/sparse setting - **no feature engineering** required!



Unsupervised Learning: Generative Adversarial Nets (GAN)



2014



2015



2016



2017



2018

NVIDIA

None of these images were taken in the real world!

Unsupervised Learning: Generative Adversarial Nets (GAN)



DeepMind - 2018

Which of these images was generated?

Unsupervised Learning: Language Generation

OpenAI, February 14 2019: Better Language Models and Their Implications

SYSTEM PROMPT
(HUMAN-WRITTEN)

In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

MODEL COMPLETION
(MACHINE-WRITTEN,
10 TRIES)

The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science.

Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved.

Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.

Pérez and the others then ventured further into the valley. "By

Unsupervised Learning: Language Generation

OpenAI, February 14 2019: Better Language Models and Their Implications

www.talktotransformer.com



So why not use Deep Learning for everything?



- ▶ There are reasons why we don't only use DL:
- Necessary data no available
- Computational power not available
- Harder to interpret results
- Deep networks can be fooled:

